

Compositional Probabilistic Optimization

*A Textbook of Process Semantics, Experimental Evidence, Plugin
Architecture, and RAG Systems*

Pedagogical edition of the ragkit / ragopt architecture study

August 2026

Contents

Front matter

- Abstract
- Preface
- Mathematical notation
- Companion implementation

Chapter 1. Processes, Probability, and Meaning

- 1.1 Begin with the system, not the optimizer
- 1.2 Categories as a language of typed processes
- 1.3 Sequential and parallel composition
- 1.4 Deterministic maps and stochastic processes
- 1.5 Copying a sampled value is not resampling a process
- 1.6 Exact finite stochastic processes as an algebraic laboratory
- 1.7 Parametrized processes: where optimization enters
- 1.8 Real systems need instrumented outcomes
- 1.9 Structured trace as intensional meaning
- 1.10 Stable randomness and compositional sampling
- 1.11 Denotational and operational semantics
- 1.12 Worked example: a compositional retrieval process
- 1.13 Common design errors
- 1.14 Chapter summary
- 1.15 Exercises

Chapter 2. Experiments as Evidence-Producing Programs

- 2.1 A metric callback is not an experiment
- 2.2 Cases and information visibility
- 2.3 Observations: the atomic evidence produced by evaluation
- 2.4 Couplings: how two stochastic systems are compared
- 2.5 Exact cell coordinates and durable custody
- 2.6 Aggregation and sufficient statistics
- 2.7 Paired differences and statistical interpretation
- 2.8 Decision semantics
- 2.9 Partial orders and Pareto fronts
- 2.10 Adaptive campaigns as stochastic control
- 2.11 Statistical games and Bayesian lenses: a useful perspective
- 2.12 Worked example: comparing two retrieval policies
- 2.13 Common design errors
- 2.14 Chapter summary
- 2.15 Exercises

Chapter 3. A Small Semantic Core and a Strong Plugin Architecture

- 3.1 The architectural problem: extensibility without semantic collapse
- 3.2 Four plugin families
- 3.3 Pure specification and effectful binding
- 3.4 Canonical specification identity
- 3.5 Typed kernels and composition combinators
- 3.6 Effects and capabilities
- 3.7 Random namespaces and trace obligations as plugin contracts
- 3.8 Law certificates and substitutability
- 3.9 Catalogs are not service locators
- 3.10 Dynamic plugins at explicit edges
- 3.11 Exact and sampled interpreters
- 3.12 Package architecture
- 3.13 Operational semantics of a cell and campaign
- 3.14 Worked example: a typed remote reranker plugin
- 3.15 Common plugin anti-patterns
- 3.16 Chapter summary
- 3.17 Exercises

Chapter 4. RAG Optimization as a Compositional Field

- 4.1 Why RAG is a demanding test domain
- 4.2 RAG as a parametrized open process
- 4.3 Domain values and authority
- 4.4 Indexing plugins
- 4.5 Query plugins
- 4.6 Answer and agent plugins
- 4.7 Evaluator plugins for RAG
- 4.8 Dependency closure and artifact reuse
- 4.9 The companion sandbox campaign
- 4.10 Running the sandbox
- 4.11 Walking through the code path
- [4.12 From the sandbox to ragkit and ragopt](#)
- 4.13 Mapping to the applied systems
- 4.14 Production refinements beyond the sandbox
- 4.15 Self-optimization without self-authorization
- 4.16 A capstone design method
- 4.17 Chapter summary
- 4.18 Exercises and capstone projects

Back matter

- Selected exercise hints
- Glossary
- Bibliography
- Companion sandbox reference

Abstract

Optimization software is often introduced through a loop: propose a configuration, run it, compute a score, and keep the best result. That picture is useful for a first script, but it becomes misleading as soon as the optimized system contains stochastic models, parallel stages, failures, hidden labels, security constraints, expensive artifacts, multiple metrics, or adaptive proposers. In a production retrieval-augmented generation system, a candidate may alter chunking, embeddings, indexes, query routing, reranking, context construction, agent behavior, or frontend presentation. The effect of that intervention is distributed across build-time and query-time processes. It cannot be understood as one untyped function from a parameter dictionary to a number.

This book develops a mathematical and software architecture for such optimization systems. The central idea is to treat the optimized system as a *composable process* and to separate four kinds of semantics:

1. **System semantics** say what deterministic and stochastic components do and how they compose.
2. **Evidence semantics** say how cases, couplings, evaluators, observations, and durable experiment cells are constructed.
3. **Decision semantics** say how hard constraints, noninferiority, target improvement, budgets, and Pareto relations turn evidence into an eligibility judgment.
4. **Control semantics** say how proposers, selectors, and adaptive campaign state choose what to evaluate next.

The mathematical backbone uses categories of processes, symmetric monoidal composition, Markov kernels and Markov categories, parametrized processes, structured traces, and labelled transition systems. These abstractions are introduced from motivation rather than assumed. Each definition is followed by concrete examples, counterexamples, software consequences, and exercises.

The software backbone is a small typed core. A plugin exposes a pure canonical specification and a separate runtime binding. The specification identifies semantic configuration, effects, capabilities, schemas, and a stable random namespace. Binding supplies clients, credentials, stores, and worker pools. Exact finite interpreters decide small algebraic laws; sampled interpreters execute realistic stochastic components with failures, resources, warnings, and traces. Evaluators and decision policies remain separate plugins so a system component cannot silently grade or select itself.

Retrieval-augmented generation is the running application. Index construction and query serving are modeled as one parametrized open process. Chunkers, representation generators, embedders, index builders, query rewriters, channel searchers, authorization filters, fusion rules, rerankers, context policies, answer generators, and agent tools are grafted onto the common backbone through typed interfaces. A companion standard-library-only Go sandbox implements the model and executes a complete paired RAG campaign.

Preface

Why this edition exists

The material behind this book began as an architectural thesis. The thesis established a broad formal model and a self-contained implementation, but it followed the order of a research argument: problem statement, theory, architecture, implementation, application, and migration. That sequence is efficient for defending a design. It is not the easiest route for a student trying to build intuition.

This edition has therefore been rewritten around a learning progression. The first chapter asks what kind of thing an optimizable system is. The second asks what counts as evidence about two such systems. The third asks how those semantics constrain interfaces and plugins. The fourth applies the complete framework to indexing and querying in RAG. Concepts are introduced only when a concrete difficulty motivates them.

The book is written for software engineers, applied researchers, and graduate students who are comfortable with typed programming and elementary probability. Familiarity with category theory is not assumed. Readers who know categories, monads, arrows, Markov kernels, Bayesian decision theory, or operational semantics will recognize the deeper structure, but the exposition builds each needed fragment from first principles.

What the book is not

This is not a general introduction to all of category theory, all of probability theory, or all of optimization. It is not a guide to one vector database, model provider, or hyperparameter-search library. It does not claim that every production component must literally be represented by a category-theory library.

The purpose of the mathematics is architectural. It tells us which distinctions must remain visible in types, specifications, traces, experiments, and decisions. The purpose of the implementation is constructive. It demonstrates that the theory can be realized in ordinary Go without a large framework or exotic runtime.

How to read the book

Chapter 1 is foundational. Readers can skim the categorical terminology on a first pass, but they should understand the difference between copying a value and resampling a process, the role of sequential and parallel composition, and the distinction between extensional outcomes and intensional traces.

Chapter 2 is the experimental core. It explains why an evaluator is a separate process, why paired experiments require explicit couplings, why failures must remain in the denominator, and why a decision policy is not the same thing as a metric.

Chapter 3 is the architectural translation. It turns the mathematical distinctions into Go interfaces, plugin factories, specifications, capabilities, law certificates, catalogs, interpreters, and durable cell state.

Chapter 4 is the complete application. It models RAG build and query behavior, defines domain plugin interfaces, explains invalidation and artifact reuse, and walks through the companion sandbox campaign.

Each chapter contains:

- a motivating problem;
- definitions in boxed callouts;
- worked mathematical examples;
- Go API signatures and pseudocode;
- counterexamples showing tempting but invalid designs;
- exercises ranging from conceptual checks to implementation projects;
- a chapter summary.

Callout conventions

Motivation. A practical problem that makes the next definition necessary.

Definition. A term whose meaning will be used precisely later.

Worked example. A complete calculation or design derivation.

Design consequence. The direct implication for APIs, storage, tests, or runtime behavior.

Counterexample. A plausible design that fails a stated law or destroys experimental validity.

Fundamentals. A side explanation of background material that can be skipped by experienced readers.

Checkpoint. A short question to test understanding before continuing.

Running example

The running system is a small RAG engine. A corpus is chunked and represented; representations are embedded and indexed; a query is rewritten, searched through lexical and vector channels, filtered by authorization, fused, optionally reranked, and converted into evidence. An evaluator knows which source documents are relevant, while the retrieval process does not. A campaign compares a baseline and several candidates under paired randomness, hard safety gates, relevance metrics, latency and index-size budgets, and a Pareto relation.

The example is intentionally small enough to calculate by hand and execute locally. Its architecture is not toy-like: it crosses the indexing/query boundary, retains failure and resource observations, separates hidden labels from system inputs, and uses the same core interfaces that a larger system would implement.

Mathematical notation

A category is written $\mathcal{C}$. Objects such as X , Y , and P denote types, schemas, or interfaces. A process from X to Y is written $f : X \rightarrow Y$. Sequential composition is $g \circ f$, meaning that f runs first. Identity on X is id_X .

Parallel composition is written $f \otimes g$. The tensor unit is I . In the concrete Go implementation, tensor is represented by typed pairs.

A probability distribution over values of type Y is written $D(Y)$. A Markov kernel from X to Y is written

$$K : X \rightarrow D(Y).$$

A parametrized process has the form

$$S : P \otimes X \rightarrow Y,$$

where P is a parameter object. A particular candidate is a value $p : I \rightarrow P$, and the instantiated process is written $S_p : X \rightarrow Y$.

The instrumented result type is abbreviated as

$$J(Y) = (Y + F + C) \otimes T \otimes R \otimes W,$$

where F is product failure, C is cancellation, T is structured trace, R is resource usage, and W is warnings or fallback information. A sampled instrumented process is therefore

$$K : X \rightarrow D(J(Y)).$$

A case object is written C_{case} when confusion with cancellation is possible. A visibility projection $v : C_{case} \rightarrow X$ hides evaluator-only labels. An evaluator is

$$E : C_{case} \otimes J(Y) \rightarrow O,$$

where O is a product-owned observation.

A campaign history is H . A proposer is $\pi : H \rightarrow D(P + Stop)$. The three decision outcomes are written

$$D = \{Eligible, Undecided, Rejected\}.$$

The notation $a \leq b$ means that b is at least as good as a under a declared preorder. It never means a universal notion of quality; the relevant metrics, directions, tolerances, and constraints must be stated.

Companion implementation

The companion probopt sandbox is a standard-library-only Go module. The principal packages are:

Package	Responsibility
core	Canonical specifications, stable seeds, outcomes, traces, sampled kernels, and composition
finite	Exact rational finite distributions and stochastic matrices
plugin	Typed factories, environments, capabilities, effect policies, law certificates, and dynamic edges
experiment	Exact case/repeat/arm coordinates, paired execution, JSONL custody, and resume
evidence	Metrics, constraints, summaries, paired differences, gates, and Pareto fronts
campaign	Candidates, builders, proposers, selectors, and immutable history
ragtoy	Complete RAG-domain graft
cmd/probopt-demo	Executable end-to-end campaign

Commands used throughout the book are:

```
go test ./...
go test -race ./...
go run ./cmd/probopt-demo -out out -seed 20260809 -repeats 5
```

The book includes the most important source fragments, but the complete sandbox is intended to be read alongside Chapters 3 and 4.

Chapter 1. Processes, Probability, and Meaning

Learning objectives

By the end of this chapter, you should be able to:

1. explain why an optimization architecture needs a semantics of the optimized system before it needs a search algorithm;
2. model deterministic and stochastic components as typed processes;
3. distinguish sequential composition from parallel composition;
4. calculate the composition of finite Markov kernels;
5. explain why copying a sampled value differs from running a random process twice;
6. use the copy and discard operations of a Markov category to reason about data flow;
7. represent an optimizable family as a parametrized process;
8. distinguish a value-only result from an instrumented outcome containing failure, trace, resources, and warnings;
9. relate denotational semantics to a small-step operational semantics;
10. identify the software contracts implied by the mathematics.

1.1 Begin with the system, not the optimizer

1.1.1 The familiar loop

A first optimization script often looks like this:

```
best = none
for candidate in candidates:
    output = run(candidate)
    score = evaluate(output)
    if best is none or score > best.score:
        best = (candidate, score)
return best
```

This loop is useful because it separates proposal, execution, evaluation, and selection at a very coarse level. The difficulty is that every interesting semantic question has been hidden inside `run` and `evaluate`.

Suppose `candidate` changes a RAG chunker. Does `run` rebuild the representations and embeddings derived from the chunks? Suppose the reranker is stochastic. Are the baseline and candidate compared under independent randomness or a shared random disturbance? Suppose the candidate times out on three difficult cases. Are those cases missing from the mean, represented as failures, or retried until they look successful? Suppose the candidate improves mean reciprocal rank but discloses an unauthorized chunk to a remote provider. Can the score compensate for that violation? Suppose an LLM proposes the next candidate after reading the holdout labels. Is the result still a valid experiment?

The loop has no vocabulary for these questions. Adding more callback arguments does not solve the underlying problem, because the issue is not only missing data. The issue is missing *structure*.

Motivation. Before we can optimize a system, we need to know what counts as the same system, how systems compose, what effects they can have, what observations an experiment records, and which transformations preserve meaning.

The central move of this chapter is to treat components as **processes with typed boundaries**. The resulting process language is small: identity, sequential composition, parallel composition, copying of deterministic values, discarding, and parametrization. Probability and instrumentation are then added without erasing those operations.

1.1.2 Four questions that a process semantics answers

A process semantics answers four architectural questions.

What are the ports? A chunker consumes a document and produces chunks. A reranker consumes a query and candidates and produces a ranking. An evaluator consumes a hidden case and an instrumented outcome and produces observations. These are different boundaries.

How are components wired? A representation generator follows a chunker. Lexical and vector search can run in parallel. An evaluator receives the system outcome but must not leak its hidden labels into the system.

What can be observed? Final output may be insufficient. We may need failures, fallback path, provider disclosure, cost, latency, artifact lineage, and random namespace.

Which rewrites are valid? Reassociating sequential composition should not change meaning. Parallel scheduling should not change semantic trace. Replacing a deterministic plugin should preserve its declared laws. Running a random plugin twice should not be confused with copying one result.

These are semantic questions. An optimization algorithm such as grid search or Bayesian optimization is downstream of them.

1.2 Categories as a language of typed processes

1.2.1 Objects and morphisms

Category theory begins from an austere observation: many systems can be understood by studying *things*, *processes* *between things*, and *composition of processes*.

Definition 1.1 (Category). A category $\mathcal{C}$ consists of:

1. a collection of **objects** $X, Y, Z, \dots$;
2. for every pair of objects, a collection of **morphisms** $f: X \rightarrow Y$;
3. an identity morphism $\text{id}_X: X \rightarrow X$ for each object;
4. a composition operation taking $f: X \rightarrow Y$ and $g: Y \rightarrow Z$ to $g \circ f: X \rightarrow Z$;
5. laws stating left identity, right identity, and associativity:

$$\text{id}_Y \circ f = f, f \circ \text{id}_X = f, h \circ (g \circ f) = (h \circ g) \circ f.$$

The words *object* and *morphism* are intentionally general. In a programming interpretation, objects can be Go types and morphisms can be total pure functions. In a database interpretation, objects can be schemas and morphisms can be queries. In a stochastic interpretation, morphisms can be Markov kernels.

The category laws are not decorative. Associativity says that the meaning of a three-stage pipeline does not depend on how its syntax is parenthesized. This is the algebraic basis for building larger systems from smaller components.

Worked example 1.1 (A deterministic text pipeline). Let

$$\begin{aligned} f &: \text{Document} \rightarrow \text{Chunks}, \\ g &: \text{Chunks} \rightarrow \text{Representations}, \\ h &: \text{Representations} \rightarrow \text{Index}. \end{aligned}$$

The composite build process is $h \circ g \circ f$. Associativity means that an implementation may first compile g and f into a reusable subpipeline or first compile h and g ; the denotation remains the same, provided the component contracts are obeyed.

1.2.2 Category laws as refactoring laws

In ordinary programming, associativity can fail observationally when hidden effects leak into the meaning. Consider a component that draws from a single mutable global random-number generator. The sequence of draws may depend on how composition is implemented, whether tracing adds a random ID, or whether a scheduler runs a sibling first. The types line up, but the process is not lawfully compositional under the intended semantics.

The architecture developed here therefore treats category laws as **plugin obligations**. A component need not be mathematically pure, but it must expose enough structure that an interpreter can preserve the relevant laws.

Design consequence. A component interface should identify semantic inputs and outputs, and its runtime should obtain randomness and effects through explicit, stable mechanisms. Hidden mutable state is not merely a testability problem; it prevents semantic substitution.

1.3 Sequential and parallel composition

1.3.1 Why sequential composition is not enough

RAG pipelines contain parallel structure. Lexical and vector retrieval can run independently from the same query. Two evaluators can score the same retained outcome. A baseline and challenger can execute at the same experimental coordinate. To describe this structure, a category needs a way to place processes side by side.

Definition 1.2 (Symmetric monoidal category). A symmetric monoidal category is a category $\mathcal{C}$ equipped with:

- a tensor product on objects, $X \otimes Y$;
- a tensor product on morphisms, taking $f: X \rightarrow X'$ and $g: Y \rightarrow Y'$ to $f \otimes g: X \otimes Y \rightarrow X' \otimes Y'$;
- a tensor unit I ;
- coherent associativity, unit, and symmetry isomorphisms.

Intuitively, $f \otimes g$ means that f and g are independent branches of one larger process graph.

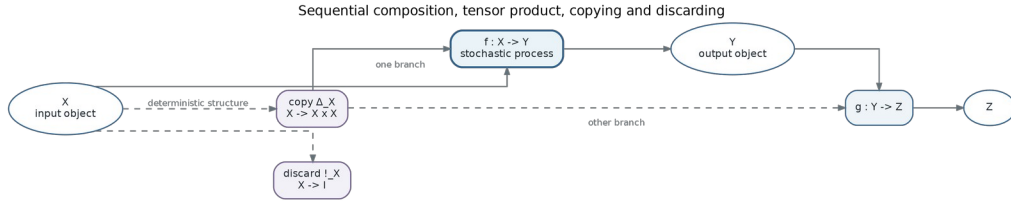
In Go, the tensor object is represented by a typed pair:

```
type Pair[A, B any] struct {
    First  A
    Second B
}
```

A tensor combinator has a signature similar to:

```
func Tensor[A, B, C, D any](
    left Kernel[A, B],
    right Kernel[C, D],
) Kernel[Pair[A, C], Pair[B, D]]
```

The mathematical tensor does not require threads. It describes *independent wiring*. One interpreter may execute the branches sequentially for determinism; another may use goroutines. The semantic trace should record a parallel node rather than accidental wall-clock order.



Sequential and parallel process structure in a Markov-category setting.

1.3.2 Fanout requires copying the input

Suppose lexical and vector search both consume the same query. Their signatures are

$$L: Q \rightarrow R_L, V: Q \rightarrow R_V.$$

Tensor alone expects an input $Q \otimes Q$. To use one query value in both branches, we need a copy process

$$\Delta_Q: Q \rightarrow Q \otimes Q.$$

Then fanout is

$$Q \xrightarrow{\Delta_Q} Q \otimes Q \xrightarrow{L \otimes V} R_L \otimes R_V.$$

The distinction between tensor and fanout matters because copying is lawful for values but not for arbitrary stochastic processes. We return to this in Section 1.5.

Checkpoint 1.1. Why can $\text{Tensor}(L, V)$ not directly have type $\text{Kernel}[\text{Query}, \text{Pair}[\text{Lexical}, \text{Vector}]]$? What additional operation is needed?

1.4 Deterministic maps and stochastic processes

1.4.1 Deterministic functions as degenerate distributions

A deterministic function $f: X \rightarrow Y$ can be viewed as a stochastic process that assigns probability one to $f(x)$. This lets deterministic and stochastic components inhabit one process category.

Definition 1.3 (Dirac distribution). For a value $y \in Y$, the Dirac distribution δ_y assigns probability one to y and zero to every other outcome.

A deterministic function f induces the kernel

$$K_f(x) = \delta_{f(x)}.$$

This embedding is important because most RAG systems mix both kinds of component. Tokenization, deterministic fusion, and authorization may be deterministic. Query rewriting, reranking, and answer generation may be stochastic.

1.4.2 Markov kernels

Definition 1.4 (Markov kernel, finite case). For finite sets X and Y , a Markov kernel $K: X \rightarrow D(Y)$ assigns to each input $x \in X$ a probability distribution over outputs Y . Equivalently, it is a row-stochastic matrix whose entries satisfy

$$K(y \mid x) \geq 0, \sum_{y \in Y} K(y \mid x) = 1.$$

The phrase *kernel* emphasizes that the output distribution depends on an input. A single distribution is the special case $I \rightarrow D(Y)$, where I has one element.

Worked example 1.2 (A noisy binary classifier). Let $X = Y = \{0, 1\}$. Define

$$K = \begin{pmatrix} 3/4 & 1/4 \\ 1/3 & 2/3 \end{pmatrix}.$$

The first row means that input 0 produces output 0 with probability 3/4 and output 1 with probability 1/4. The second row means that input 1 produces output 0 with probability 1/3 and output 1 with probability 2/3.

This example appears in the sandbox exact-law certificate:

```
coin := finite.Kernel[bit, bit]{
  Name: "coin",
  Run: func(b bit) finite.Dist[bit] {
    if b == zero {
      return finite.Dist[bit]{
        zero: finite.NewProb(3, 4),
        one:  finite.NewProb(1, 4),
      }
    }
    return finite.Dist[bit]{
      zero: finite.NewProb(1, 3),
      one:  finite.NewProb(2, 3),
    }
  },
}
```

1.4.3 Composing kernels

If $K : X \rightarrow D(Y)$ and $L : Y \rightarrow D(Z)$, their composite marginalizes the intermediate value:

Definition 1.5 (Composition of finite Markov kernels).

$$(L \circ K)(z \mid x) = \sum_{y \in Y} L(z \mid y) K(y \mid x).$$

This is ordinary matrix multiplication with probabilities interpreted conditionally.

Worked example 1.3 (Noisy classifier followed by deterministic flip). Let F flip a bit:

$$F = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

For input 0,

$$(F \circ K)(0 \mid 0) = F(0 \mid 0)K(0 \mid 0) + F(0 \mid 1)K(1 \mid 0) = 0 \cdot \frac{3}{4} + 1 \cdot \frac{1}{4} = \frac{1}{4}.$$

Similarly $(F \circ K)(1 \mid 0) = 3/4$. The deterministic flip merely permutes the output probabilities.

1.4.4 Why a monad alone does not settle the architecture

Probability distributions form a monadic pattern: a function $X \rightarrow D(Y)$ can be composed using `bind`. That observation is useful, but an optimization architecture also needs to discuss copying, discarding, parallel composition, and deterministic substructure. Markov-category language makes those operations central rather than incidental (Fritz 2020).

Fundamentals: monads and Kleisli composition. A monad packages a type constructor and operations for injecting pure values and sequencing effectful computations. The category of Kleisli arrows has morphisms $X \rightarrow T(Y)$. Markov kernels fit this pattern when T is a probability-distribution construction. Markov categories add a symmetric monoidal process view with explicit copy and discard structure.

1.5 Copying a sampled value is not resampling a process

This is the most important conceptual distinction in the chapter.

Suppose a fair coin process is

$$C : I \rightarrow D(B), B = \{H, T\}.$$

There are two ways to obtain a pair of bits.

Sample once, then copy the value:

$$I \xrightarrow{C} B \xrightarrow{\Delta_B} B \otimes B.$$

The result distribution is

$$P(H, H) = 1/2, P(T, T) = 1/2,$$

with zero probability for mixed pairs.

Run the coin process twice independently:

$$I \cong I \otimes I \xrightarrow{C \otimes C} B \otimes B.$$

The result distribution is uniform over all four pairs.

These processes have the same output type but different meanings.

Counterexample 1.1 (Invalid stochastic copying law). The equation

$$\Delta_B \circ C = (C \otimes C) \circ \Delta_I$$

is false for a non-deterministic coin. If an API silently treats fanout of a random component as two independent executions, it changes correlation and can invalidate paired experiments.

1.5.1 Markov categories

Definition 1.6 (Markov category, engineering fragment). A Markov category is a symmetric monoidal category in which every object X has:

- a copy map $\Delta_X : X \rightarrow X \otimes X$;
- a discard map $!_X : X \rightarrow I$;

satisfying commutative comonoid laws and compatibility with deterministic morphisms. General stochastic morphisms need not preserve copying.

Worked example 1.4a (Two uses of copy). A query value can be copied so lexical and vector search receive the same text. A complete evaluation case can also be copied so one branch projects the visible query while the evaluator branch retains hidden relevant-document labels. In both situations a deterministic value is shared; no stochastic component is rerun.

Discard says that a process may ignore a value. Causality says that a normalized stochastic process followed by discard is the same as discarding its input:

$$!_Y \circ K = !_X.$$

Deterministic maps preserve copy:

$$\Delta_Y \circ f = (f \otimes f) \circ \Delta_X.$$

The failure of this law for general stochastic processes precisely captures the sample-versus-resample distinction.

1.5.2 Why copy and discard matter in experiments

Copying a deterministic case allows one branch to provide visible input to the system while another branch retains hidden labels for evaluation. Discarding permits an evaluator to ignore unneeded trace fields or a decision rule to project only selected metrics. The laws let us reason that these operations do not alter the branches they do not inspect.

Later, one trial will have the form

$$C_{case} \xrightarrow{\Delta} C_{case} \otimes C_{case} \xrightarrow{id \otimes v} C_{case} \otimes X \xrightarrow{id \otimes S_p} C_{case} \otimes J(Y) \xrightarrow{E} O.$$

The copy occurs on a deterministic case value, not on the stochastic system process.

1.6 Exact finite stochastic processes as an algebraic laboratory

1.6.1 Why build an exact interpreter?

Production systems expose only samples. A test that samples the same process many times can estimate whether two distributions are close, but it cannot prove exact equality, and failures may be flaky. Small finite stochastic systems permit exact rational calculation.

The sandbox package `finite` implements a practical fragment of the category often called *FinStoch*:

- objects are finite Go types;
- morphisms are exact rational stochastic matrices;
- composition is exact marginalization;
- tensor is product distribution;
- copy and discard are deterministic kernels.

This exact model is not the production runtime. It is a **law oracle** and teaching tool.

1.6.2 Exact probabilities

A probability is stored as a normalized rational number. The distribution type is a finite map:

```
type Dist[T comparable] map[T]Prob
```

A kernel is:

```
type Kernel[A, B comparable] struct {
    Name string
    Run func(A) Dist[B]
}
```

Composition has the direct mathematical implementation:

```
func Compose[A, B, C comparable](
    left Kernel[A, B],
    right Kernel[B, C],
) Kernel[A, C] {
    return Kernel[A, C]{
        Name: right.Name + " o " + left.Name,
        Run: func(a A) Dist[C] {
            out := Dist[C]{}
            for b, pAB := range left.Run(a) {
                for c, pBC := range right.Run(b) {
                    out[c] = out[c].Add(pAB.Mul(pBC))
                }
            }
            return out
        },
    }
}
```

1.6.3 Executable laws

The demo certifies:

- left identity;
- right identity;
- associativity;
- discard naturality;
- deterministic copy naturality;
- copy commutativity.

A law checker enumerates a finite input set and compares exact distributions. This makes the algebra concrete: the software can reject a broken interpreter before any RAG experiment runs.

Worked example 1.4 (Associativity). Let $K : X \rightarrow D(Y)$, $L : Y \rightarrow D(Z)$, and $M : Z \rightarrow D(W)$. For input x ,

$$\underbrace{\left((M \circ L) \circ K \right)}_{\text{⋈}}(w \mid x) \stackrel{\text{⋈}}{=} \sum_y \left(\underbrace{\sum_z M(w \mid z) L(z \mid y)}_{\text{⋈}} \right) K(y \mid x)$$

while

$$\underbrace{(M \circ (L \circ K))}_{\text{⋈}}(w \mid x) \stackrel{\text{⋈}}{=} \sum_z M(w \mid z) \left(\underbrace{\sum_y L(z \mid y) K(y \mid x)}_{\text{⋈}} \right)$$

Finite sums can be reordered, so the two expressions are equal.

1.6.4 The limits of the finite model

The exact interpreter does not model continuous distributions, external providers, timeouts, streaming, or unbounded state. It should not be stretched into a production probability engine. Its role is to provide a clear semantics for composition and a place where laws can be decided exactly.

Design consequence. Maintain at least two interpreters for important semantic signatures: a simple reference interpreter whose behavior is easy to reason about, and a production interpreter that adds concurrency, batching, caching, and external effects. Test the production interpreter against the reference relation.

1.7 Parametrized processes: where optimization enters

1.7.1 A family is not a bag of callbacks

An optimizer does not usually choose a completely unrelated program on every trial. It chooses a value in a structured parameter space and instantiates a family of systems. The family itself should therefore have a type.

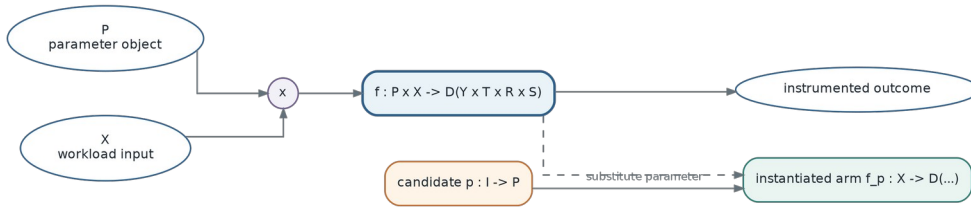
Definition 1.7 (Parametrized process). A parametrized process from X to Y consists of a parameter object P and a process

$$S : P \otimes X \rightarrow Y.$$

A particular candidate is a global element $p : I \rightarrow P$. Instantiating S at p produces

$$S_p : X \rightarrow Y.$$

This is the practical fragment of the categorical Para construction used in compositional learning theory (Cruttwell et al. 2022). No differentiability is required. P may contain numbers, enums, structured policies, model identities, immutable artifact references, or tagged alternatives.



A parametrized process separates the family from one selected candidate.

1.7.2 A RAG parameter object

A simplified RAG parameter type might be:

```
type Params struct {
    ChunkWords      int
    ChunkOverlap    int
    Representation   RepresentationKind
    EmbeddingModel  ModelID
    LexicalWeight   float64
    VectorWeight    float64
    CandidateDepth  int
    RerankDepth     int
    Reranker        *ModelID
    EvidenceLimit   int
}
```

This is already better than `map[string]any` because it makes impossible combinations easier to reject and exposes the factorization of the system. It is still only a data type. The semantic family is the process that consumes it:

$$S : Params \otimes CaseInput \rightarrow J(SystemOutput).$$

A candidate should also carry identity, parent, hypothesis, and locked assets, but those are campaign metadata rather than the parameter object itself.

1.7.3 Composition accumulates parameters

Suppose index construction and query serving are separate families:

$$B : P_B \otimes Corpus \rightarrow Release,$$

$$Q:P_Q \otimes (\textit{Release} \otimes \textit{Request}) \rightarrow J(\textit{Answer}).$$

Their composite family has parameter object $P_B \otimes P_Q$. A query-only candidate fixes p_B and varies p_Q . A chunking candidate varies p_B and requires a new release. A joint candidate varies both.

This factorization is not only mathematical elegance. It induces artifact invalidation. Changing a query fusion weight does not invalidate embeddings. Changing an embedding model does. The dependency graph used by a production optimizer is a finite software presentation of the compositional parameter structure.

Worked example 1.5 (Invalidation by parameter position). Consider

$$\textit{Corpus} \xrightarrow{\textit{Chunk}_{p_c}} \textit{Chunks} \xrightarrow{\textit{Embed}_{p_e}} \textit{Vectors} \xrightarrow{\textit{Build}} \textit{Index} \xrightarrow{\textit{Search}_{p_s}} \textit{Hits}.$$

- Changing p_q re-executes only `Search` and downstream evaluation.
- Changing p_e invalidates vectors, the vector index, search outcomes, and downstream evaluation, but not chunk records.
- Changing p_c invalidates chunks and every downstream artifact.

The invalidation closure is obtained from the process graph rather than a hand-maintained list of filenames.

1.7.4 Reparametrization

Sometimes a product exposes a simpler parameter space P' and translates it into a detailed internal space P through $r:P' \rightarrow P$. The reparametrized system is

$$S \circ (r \otimes id_X): P' \otimes X \rightarrow Y.$$

This formalizes presets, constrained search spaces, and backwards-compatible configuration adapters. The translation itself has semantics and identity; it should not be an invisible JSON rewrite.

Counterexample 1.2 (Configuration aliasing). Suppose the public value `balanced` maps to `{lexical: 1, vector: 1, depth: 40}` today and `{lexical: 0.8, vector: 1.2, depth: 60}` next month. If campaign history stores only `balanced`, the candidate is not reproducible. The reparametrization revision or fully expanded canonical parameter value must be retained.

1.8 Real systems need instrumented outcomes

1.8.1 Why $X \rightarrow D(Y)$ is still too small

A stochastic output value does not capture production behavior. A reranker can fail and fall back to fused order. A query can be cancelled. A model can return a valid answer after three retries. A candidate can expose source text to a remote provider, consume more tokens, or use a different artifact release. These facts may be invisible in Y but decisive for safety and optimization.

Definition 1.8 (Instrumented outcome). For output type Y , an instrumented outcome contains:

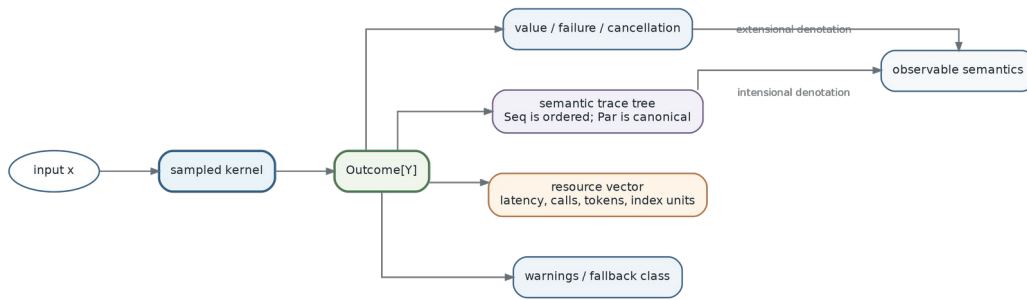
- either a value Y , product failure F , or cancellation C ;
- a structured trace T ;
- a resource observation R ;
- warnings or fallback information W .

We abbreviate the result object as

$$J(Y) = (Y + F + C) \otimes T \otimes R \otimes W.$$

A sampled component therefore has denotation

$$K: X \rightarrow D(J(Y)).$$



An instrumented stochastic kernel retains value, failure, trace, resources, and warnings.

1.8.2 Product failure versus interpreter failure

The Go sandbox makes a strict distinction:

```

type Outcome[T any] struct {
    Value      *T
    Status     Status
    Failure    *Failure
    Warnings   []string
    Trace      Trace
    Resources  Resources
}
  
```

A product failure such as `reranker_timeout`, `invalid_answer_contract`, or `no_authorized_evidence` is an Outcome. It occupies an experimental cell and remains in the denominator. An ordinary Go error is reserved for failure of the execution contract itself, such as a corrupt ledger, a bound plugin with the wrong specification, or inability to persist the cell.

This distinction prevents a common bias: treating difficult product failures as if the experiment never happened.

Counterexample 1.3 (Retry until success). A candidate has a 20% timeout rate. The runner retries each timeout until it obtains a success and records only the successful metric. The reported quality distribution is conditioned on eventual success and cannot be compared with a baseline that succeeded on the first attempt. Timeouts must be recorded as outcomes or as explicit attempts within the outcome trace.

1.8.3 Status is not a Boolean

The sandbox uses four statuses:

```

const (
    StatusCompleted Status = "completed"
    StatusDegraded   Status = "degraded"
    StatusFailed     Status = "failed"
    StatusCancelled  Status = "cancelled"
)
  
```

Degraded is important for systems with declared fallback. A remote reranker may time out while local hybrid retrieval remains usable. Treating this as ordinary success hides reliability regression; treating it as total failure may be too strict for serving semantics. The explicit status lets evaluation and gates decide.

1.8.4 Resource observations form an algebra

A resource map might contain:

```

latency_ms      -> 18.7
provider_calls  -> 2
input_tokens    -> 1460
index_units     -> 688
remote_bytes    -> 9214
  
```

The reference sandbox adds resources under both sequential and parallel composition. This is appropriate for additive cost and call counts but only a model for wall-clock latency. A production resource schema should declare the composition law of each dimension:

- additive for money, tokens, energy, and provider calls;
- maximum or critical path for parallel elapsed time;
- maximum for peak memory;
- set union for disclosed data classes;
- sum with deduplication for bytes retained in shared artifacts.

Definition 1.9 (Resource algebra). A resource algebra is a set R with one or more composition operations describing how resource observations combine under process wiring. The simplest case is a commutative monoid $(R, +, 0)$.

The key lesson is that “usage” is not one scalar. Its algebra depends on the dimension and the process structure.

1.9 Structured trace as intensional meaning

1.9.1 Extensional and intensional equality

Two systems are **extensionally equal** under a projection when they produce the same observable values or value distributions. They are **intensionally equal** only when relevant internal facts also agree.

Consider two query systems that return the same ranked document IDs. The first filters unauthorized candidates before local reranking. The second sends all candidate text to a remote reranker and filters afterward. Their returned values may match. Their disclosure traces do not. For a security comparison, extensional equality of rankings is insufficient.

Definition 1.10 (Semantic trace). A semantic trace is a structured record of intensional events that are declared relevant to the meaning or comparison of a process: component identities, branches, fallbacks, disclosures, lineage, retries, and similar facts. It excludes incidental scheduler order unless order itself is semantically relevant.

1.9.2 A trace algebra

Sequential composition forms an ordered node:

$$T_{g \circ f} = Seq(T_f, T_g).$$

Parallel composition forms a symmetric or canonically ordered node:

$$T_{f \otimes g} = Par(T_f, T_g).$$

A leaf contributes an event labeled by specification identity and structured fields. The sandbox trace type is:

```
type Trace struct {
  Kind      TraceKind
  Label     string
  SpecID    string
  Fields    map[string]string
  Children  []Trace
}
```

A production trace can store large payloads as content-addressed references. The semantic tree should not depend on goroutine completion order.

1.9.3 Trace projections

Not every consumer may observe every trace field. A customer frontend may receive safe source lineage; an administrator may receive provider and fallback information; an evaluator may receive exact timing and native artifacts; a proposer may receive only aggregate metrics.

This suggests typed projections

$$\pi_i: T \rightarrow T_i$$

with information-flow policy. Trace existence does not imply universal visibility.

Design consequence. Treat traces as versioned domain data, not log strings. Define event schemas, visibility classes, canonical composition, and artifact references. Logs may render traces; they should not be the authoritative trace format.

1.10 Stable randomness and compositional sampling

1.10.1 The mutable-RNG associativity trap

Suppose $\text{Compose}(\text{Compose}(f, g), h)$ and $\text{Compose}(f, \text{Compose}(g, h))$ both share one mutable RNG. If the combinator adds bookkeeping draws or changes scheduling, the leaf components may receive different random numbers. The sampled result then depends on syntax.

The sandbox avoids this by deriving each leaf stream from a root seed and a stable semantic namespace:

```

type Seed [32]byte

func (s Seed) For(spec Spec, labels ...string) Seed {
    ns := spec.RandomNamespace
    if ns == "" {
        ns = spec.ID
    }
    return s.Derive(append([]string{"component", ns}, labels...)...)
}

```

The same root seed passes through the whole process tree. A leaf derives its own stream from its namespace and experiment coordinates. Reparenthesizing composition does not change the leaf stream.

Definition 1.11 (Stable random namespace). A stable random namespace is a semantic identifier used with a root seed and coordinate labels to derive a component-local pseudorandom stream. Distinct stochastic stages use distinct namespaces; deliberately coupled implementations may share one.

1.10.2 Correlation is design, not an accident

If baseline and candidate rerankers should receive the same random perturbation, they can share a semantic namespace at the same case/repeat coordinate. If they should be independent, they use distinct coupling labels. This will become central in Chapter 2.

1.10.3 Randomness and implementation identity

The random namespace is not necessarily the component spec ID. Two candidate implementations may have different specs but intentionally share a namespace to implement common random numbers. Conversely, two stochastic stages with the same model must not share a namespace if their draws represent different semantic calls.

Counterexample 1.4 (One namespace per model). A query-rewrite call and answer-generation call both use model M and namespace model-M . Their draws become correlated and may collide under identical labels. Namespaces should identify semantic call families, such as *rewrite/query-variants* and *answer/final-generation*, not only provider identity.

1.11 Denotational and operational semantics

1.11.1 Two complementary views

A denotational semantics maps a specification to a mathematical process. It answers: *what distribution over instrumented outcomes does this component denote?*

An operational semantics describes execution steps. It answers: *how does a runtime reach one sampled outcome, including short circuit, cancellation, binding, ledger commit, and scheduling?*

Neither view replaces the other.

Definition 1.12 (Denotational semantics). Given a realization environment ρ , the denotation of a typed specification $s : X \rightarrow Y$ is a process

$$\llbracket s \rrbracket_{\rho} \in C(X, J(Y)).$$

The interpretation preserves identity, sequential composition, and tensor.

Definition 1.13 (Operational semantics). An operational semantics is a transition or evaluation relation over runtime configurations. A sampled configuration can be written

$$\langle K, x, s, m \rangle,$$

where K is a kernel, x input, s root seed, and m operational metadata.

Worked example 1.6 (Cache semantics). Denotationally, a verified cache hit for a deterministic embedder denotes the same vector as fresh execution. Operationally, the cache interpreter performs lookup, digest verification, and either returns the retained vector or executes the provider path. The two operational traces differ, but a projection that ignores cache mechanics and preserves vector, model identity, failure, and disclosure can treat them as equivalent.

1.11.2 Sequential operational rule

For $K = g \circ f$:

1. run f with root seed s ;

2. if f has no value, return its failure or cancellation with a short-circuit trace;
3. otherwise run g on the produced value using the same root seed;
4. combine status, warnings, trace, and resources.

In inference-rule notation:

$$\frac{\rho \vdash \langle f, x, s, m \rangle \Downarrow o_1 = \text{Success}(y, t_1, r_1) \quad \rho \vdash \langle g, y, s, m \rangle \Downarrow o_2}{\rho \vdash \langle g \circ f, x, s, m \rangle \Downarrow \text{combine}_{seq}(o_1, o_2)}.$$

If the first process fails:

$$\frac{\rho \vdash \langle f, x, s, m \rangle \Downarrow o_1 \text{ value}(o_1) = \emptyset}{\rho \vdash \langle g \circ f, x, s, m \rangle \Downarrow \text{short}(o_1, g)}.$$

1.11.3 Parallel operational rule

For tensor:

$$\frac{\rho \vdash \langle f, x_1, s, m \rangle \Downarrow o_1 \quad \rho \vdash \langle g, x_2, s, m \rangle \Downarrow o_2}{\rho \vdash \langle f \otimes g, (x_1, x_2), s, m \rangle \Downarrow \text{combine}_{par}(o_1, o_2)}.$$

The premises may execute in either order or concurrently. Stable namespaces and canonical parallel trace make the semantic result independent of schedule, subject to plugin laws.

1.11.4 Refinement between interpreters

A production interpreter may batch embeddings, cache deterministic outputs, execute channels concurrently, or call remote services. It is a valid refinement of the reference semantics only under a declared relation. Exact equality may be too strong when timing differs and too weak when disclosure is projected away.

Definition 1.14 (Refinement under a projection). Let $\pi : J(Y) \rightarrow V$ select the observations relevant to a claim.

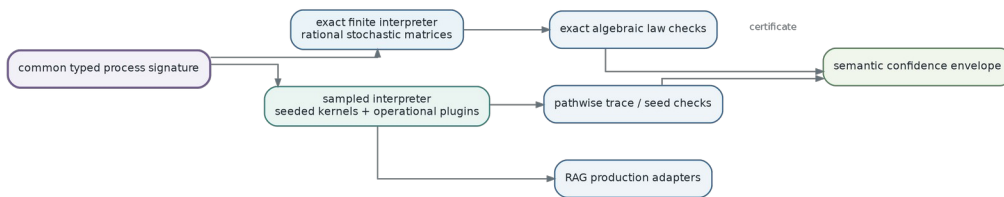
An implementation K' refines K under relation $\leq_V$ when, for every allowed input, their projected outcome distributions satisfy the relation.

Examples include:

- exact equality of ranked outputs;
- pathwise equality under shared random seeds;
- distributional noninferiority;
- authorization noninterference;
- lower resource use with identical value and failure behavior.

“Equivalent” is incomplete unless the projection and relation are named.

Worked example 1.7 (ANN refinement). An approximate vector index does not refine exact search under equality of ranks. It may refine it under the projection (authorized IDs, recall@k, latency) and relation “all returned IDs are authorized, recall@k is at least 0.95, and p95 latency is no greater.” The exact index remains the oracle in contexts that require exact neighbors.



One semantic signature can have exact reference and sampled production interpreters.

1.12 Worked example: a compositional retrieval process

We now assemble the concepts in a small retrieval example.

1.12.1 Types

Let:

```
type Query struct {
  Text    string
  Subject Subject
}
```

```

type Candidate struct {
    DocumentID string
    Score      float64
    Scope      string
}

type Hits struct {
    Items []Candidate
}

```

The component signatures are:

$$\begin{aligned}
 L & : Query \rightarrow J(R_L), \\
 V & : Query \rightarrow J(R_V), \\
 A & : (Query \otimes (R_L \otimes R_V)) \rightarrow J(R_A), \\
 F & : R_A \rightarrow J(Hits).
 \end{aligned}$$

L and V search lexical and vector indexes. A applies authorization before any remote stage. F fuses the authorized rankings.

1.12.2 Process graph

Fan out the query, search both channels, preserve one query copy for authorization, then fuse:

$$Query \xrightarrow{\Delta^{[3]}} Query \otimes Query \otimes Query \xrightarrow{id \otimes L \otimes V} Query \otimes R_L \otimes R_V \xrightarrow{A} R_A \xrightarrow{F} Hits.$$

A is downstream of local search but upstream of any remote reranker. Its trace can certify which subject and policy revision authorized each candidate.

1.12.3 Parametrization

Let

$$P = N_{i_0} \otimes R_{\geq 0}^2$$

contain candidate depth and lexical/vector weights. Fusion becomes

$$F : P \otimes R_A \rightarrow Hits.$$

A candidate $p = (40, 1.0, 1.3)$ instantiates one retrieval policy. An optimizer can vary p without rebuilding indexes. If a candidate also changes embeddings, the parameter object and process graph grow to include build parameters.

1.12.4 Instrumented outcomes

Suppose vector search times out. The process may return a degraded outcome containing lexical hits, warning `vector_timeout`, a trace that records the skipped vector contribution, and resources showing one failed provider call. The final hits may be acceptable, but the experiment can gate degradation rate.

Suppose authorization fails to load its policy. Returning unfiltered hits would violate the process contract. The outcome is a product failure or interpreter failure depending on whether policy unavailability is a declared domain state. It is not a fallback to global retrieval.

1.12.5 Stable seed

If fusion includes stochastic tie-breaking, it derives its randomness from:

```

root seed
+ case ID
+ repeat number
+ semantic namespace "retrieval/fusion-tie-break"

```

Changing parallel scheduling does not change the tie. A baseline and candidate may share the namespace to compare under the same tie disturbance.

1.12.6 What the optimizer still does not know

At this point we have a system family, but no experiment. The system does not know which documents are relevant. It does not know whether latency is acceptable. It does not decide whether a candidate should be promoted. Those belong to the evidence and decision semantics of Chapter 2.

1.13 Common design errors

Error 1: one untyped process interface

```
type Plugin interface {
    Run(context.Context, map[string]any) (map[string]any, error)
}
```

This interface can encode every component and therefore communicates almost nothing. It erases ports, hidden-label separation, effects, random namespaces, and output laws. It may be acceptable at an external dynamic boundary, but it should be immediately adapted to a typed internal process.

Error 2: copy by rerunning

A fanout combinator implemented by calling a stochastic upstream component twice produces independent samples rather than two references to one sample. This changes correlation.

Error 3: trace as logging side effect

If trace events are emitted only to process logs, a resumed experiment cannot prove which fallback or artifact produced a stored metric. The trace must be part of the retained outcome or referenced native artifact.

Error 4: all failures are Go errors

If ordinary product failures abort the runner, missing difficult cases bias the experiment and prevent paired comparison. Use typed outcome status for expected domain failures.

Error 5: operational identity contaminates semantic identity

Worker count, temporary directory, timestamp, and hostname usually should not change a component spec ID. Model version, prompt bytes, fallback policy, and disclosure class usually should. Mixing them causes either unnecessary cache misses or unsound reuse.

1.14 Chapter summary

An optimizable system should be modeled before its optimizer. Categories supply typed processes, identities, and associative sequential composition. Symmetric monoidal structure adds parallel wiring. Markov kernels model stochastic components; Markov-category copy and discard distinguish sharing a sampled value from resampling a process. Exact finite kernels provide an executable algebraic oracle.

Parametrized processes separate a system family from one candidate. Instrumented outcomes retain failures, cancellation, traces, resources, and warnings. Stable random namespaces make sampling compositional. Denotational semantics states what a specification means; operational semantics states how an interpreter executes it. Production implementations are refinements under explicitly named projections and relations.

The remaining problem is epistemic: how do we obtain trustworthy evidence that one instantiated process is better than another? That is the subject of Chapter 2.

1.15 Exercises

Conceptual exercises

1. **Function category.** Verify the three category laws for ordinary pure functions. Which assumptions about functions are required?
2. **Kernel composition.** Let

$$K = \begin{pmatrix} 1/2 & 1/2 \\ 1/4 & 3/4 \end{pmatrix}, L = \begin{pmatrix} 1 & 0 \\ 1/3 & 2/3 \end{pmatrix}.$$

Compute $L \circ K$.

3. **Copy versus resample.** For a biased coin with $P(H) = p$, write the joint distribution for sample-then-copy and sample-twice. For which values of p are the distributions equal?

4. **Discard naturality.** Explain in plain language why $!_Y \circ K = !_X$ expresses normalization of a stochastic process.
5. **Parametrization.** Model a three-option reranker choice as a parameter object. How does a candidate instantiate the family?
6. **Invalidation.** Draw a process graph for normalization, chunking, summary generation, embedding, vector indexing, search, and reranking. Mark the invalidation closure of a summary-prompt change.
7. **Outcome design.** Classify each event as product failure, degradation, cancellation, or interpreter failure: empty authorized evidence; remote timeout with local fallback; corrupt cell ledger; user cancellation; invalid model JSON repaired successfully; artifact digest mismatch.
8. **Trace projection.** Give three trace fields that a product evaluator may need but a customer frontend should not receive.

Mathematical exercises

9. **Associativity by sums.** Prove associativity of finite Markov-kernel composition by expanding both sides and reordering finite sums.
10. **Deterministic copy naturality.** Let $f : X \rightarrow Y$ be a function embedded as a deterministic kernel. Prove

$$\Delta_Y \circ f = (f \otimes f) \circ \Delta_X.$$
11. **Stochastic counterexample.** Construct a non-deterministic kernel for which copy naturality fails, and calculate both sides explicitly.
12. **Tensor composition.** Prove that product kernels satisfy

$$(f_2 \circ f_1) \otimes (g_2 \circ g_1) = (f_2 \otimes g_2) \circ (f_1 \otimes g_1).$$
13. **Resource algebra.** Propose a product resource algebra containing cost, elapsed time, peak memory, and disclosed data classes. Define sequential and parallel composition for each field.
14. **Refinement relation.** Define a projection and preorder under which a local deterministic reranker can refine a remote stochastic reranker even when their exact rankings are not always equal.

Programming exercises

15. **Finite kernel.** Implement a rational finite kernel type in a language of your choice and check left/right identity over a finite input set.
16. **Stable seeds.** Implement a domain-separated seed derivation function. Demonstrate that parallel branch scheduling does not alter leaf draws.
17. **Trace tree.** Implement Seq and Par trace nodes. Make Par canonical under branch order without erasing branch identity.
18. **Instrumented compose.** Implement sequential composition that short-circuits failure, preserves trace, and combines resources.
19. **Specification audit.** Take a real component configuration from your system. Separate semantic fields from operational binding fields and justify each classification.
20. **RAG graph.** Implement the small lexical/vector/authorization/fusion process from Section 1.12 using typed composition. Add a test that unauthorized candidate text never reaches a mock remote reranker.

Research and design exercises

21. Compare the Markov-category process view with a monadic API and an Arrow-style API. Which structure is easiest to inspect statically in your implementation language?
22. Investigate partial Markov categories. How might subprobability or partiality change the representation of failure used here?
23. Define a notion of trace equivalence that ignores cache hits and worker scheduling but preserves disclosure, artifact lineage, and fallback class.
24. Design a reference/production interpreter pair for one component in your current system. State the refinement relation that should hold between them.

Chapter 2. Experiments as Evidence-Producing Programs

Learning objectives

By the end of this chapter, you should be able to:

1. explain why evaluation must be modeled separately from the system under test;
2. split a case into system-visible input and evaluator-only material;
3. define a coupling of two stochastic systems and verify its marginal laws;
4. design paired experiments with stable case, repeat, and arm coordinates;
5. preserve failures, missingness, and native artifacts in a durable cell ledger;
6. distinguish raw observations, mergeable summaries, statistical analyses, and decision rules;
7. formulate hard constraints, noninferiority, target improvement, and resource budgets;
8. explain why many optimization decisions form a partial order rather than a total order;
9. model an adaptive campaign as a state transition over immutable history;
10. identify information-flow rules that prevent holdout leakage and self-grading.

2.1 A metric callback is not an experiment

2.1.1 The hidden ambiguity

A common API asks the system to return a metric map:

```
type Arm interface {
    Run(context.Context, Case) (map[string]float64, error)
}
```

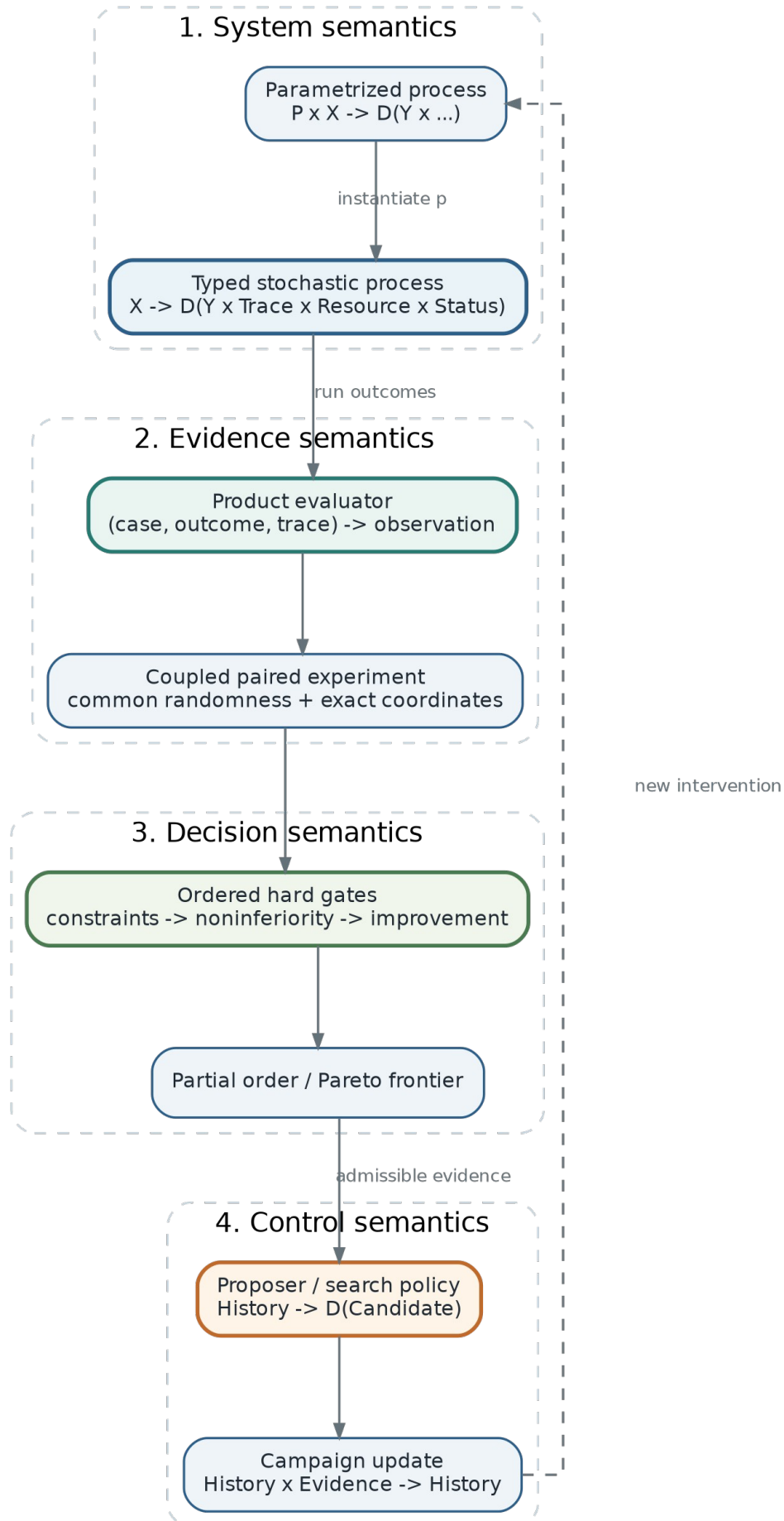
This design seems convenient because one callback can perform everything. It can load a candidate, execute the system, compare output with labels, call an LLM judge, compute cost, and return numbers. The convenience is precisely the problem. The callback controls both the behavior being evaluated and the standard by which it is evaluated.

The following questions become unanswerable from the outside:

- Which fields of the case were visible to the system?
- Did the candidate read its expected answer?
- Did the evaluator score a failed output or drop it?
- Were baseline and candidate coupled under the same stochastic disturbance?
- Did the candidate choose its own judge prompt?
- Which native trace supports the reported metric?
- Are absent metrics zero, missing, or silently ignored?
- Did adaptive proposal inspect holdout details?

Motivation. An experiment must be a composition of independent semantic roles: system execution, evaluator observation, evidence aggregation, decision policy, and adaptive control. Combining them into one callback prevents causal attribution.

The four-layer doctrine introduced in the preface is shown in Figure 2.1.



System, evidence, decision, and control semantics are separate compositional layers.

2.2 Cases and information visibility

2.2.1 A case contains more than an input

A retrieval case may contain a query, subject, expected relevant document IDs, forbidden sources, tags, and an answer rubric. The system should see the query and server-owned subject context. It should not see expected document IDs or the judge rubric.

Definition 2.1 (Case and visibility projection). A case object C_{case} contains all material required to execute and evaluate one workload item. A deterministic visibility projection

$$v: C_{case} \rightarrow X$$

extracts the input X that the system is permitted to observe.

A simple Go representation is:

```
type Case struct {
    ID          string
    Query       Query
    ExpectedDocumentIDs []string
    ForbiddenDocumentIDs []string
    Tags        []string
}

func (c Case) Visible() Query { return c.Query }
```

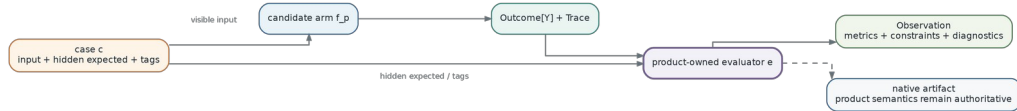
The separation should be enforced structurally. Passing the whole Case to the system and relying on comments not to read labels is weak information-flow control.

2.2.2 One trial as a process diagram

Copy the deterministic case. Keep one copy for the evaluator, and project the other to visible input:

$$C_{case} \xrightarrow{\Delta} C_{case} \otimes C_{case} \xrightarrow{id \otimes v} C_{case} \otimes X \xrightarrow{id \otimes S_p} C_{case} \otimes J(Y) \xrightarrow{E} O.$$

The system S_p never receives the full case. The evaluator E receives both hidden material and the instrumented outcome.



The evaluator is a separate process with access to hidden case material and the system outcome.

Definition 2.2 (Evaluator). An evaluator is a product-owned process

$$E: C_{case} \otimes J(Y) \rightarrow D(O)$$

that converts a hidden case and instrumented system outcome into an observation O . The evaluator may itself be deterministic or stochastic.

2.2.3 Why the evaluator is product-owned

A generic optimization kernel does not know whether a Garden product card is correct, whether a GEC answer uses an authoritative procedure, whether a RAG-TTC tool loop chose an acceptable action, or whether a retrieval miss is severe. Those meanings live in product cases, rubrics, validators, and native artifacts.

The shared core should own the *shape* of evidence custody: exact coordinates, evaluator identity, hidden-label separation, finite metrics, constraints, artifacts, and resume. The product evaluator owns the meaning of observations.

Design consequence. Shared evidence should be a comparison projection. The native product artifact remains the diagnostic authority. A metric such as groundedness=0.8 is not enough to explain which claim failed or which source was unsupported.

2.2.4 Self-grading is invalid even when honest

A system plugin may contain useful internal validation. A grounded-answer component can check whether cited IDs were in the admitted evidence. That check is part of system semantics and can become a hard constraint observation. It is not a complete evaluator, because it cannot independently judge whether the admitted evidence was relevant, whether the answer omitted required content, or whether the system manipulated its own validator.

Counterexample 2.1 (Candidate-owned judge). A candidate changes both an answer prompt and the judge prompt that scores the answer. It reports a large improvement. Even if the judge change was intended to be more accurate, the experiment no longer isolates the answer intervention. The evaluator must be locked or the two changes must be treated as one explicitly declared intervention with independent external validation.

2.3 Observations: the atomic evidence produced by evaluation

2.3.1 Observation schema

Definition 2.3 (Observation). An observation is the evaluator’s typed result for one completed system execution. It may contain:

- finite numeric metrics with declared directions and units;
- Boolean or categorical constraints;
- tags and strata;
- failure and degradation classification;
- resource observations;
- references to native artifacts and traces;
- evaluator warnings and diagnostics.

One possible Go schema is:

```
type Observation struct {
    Metrics      map[string]float64
    Constraints  map[string]bool
    Tags         []string
    Status       string
    Native       ArtifactRef
    Trace        ArtifactRef
}
```

The actual sandbox uses small structs around metrics and constraints. A production schema should additionally carry metric specifications, evaluator version, and artifact identity.

2.3.2 Metrics have direction and domain

A metric name without direction is incomplete. Higher recall is better; lower latency is better. Some metrics are bounded in $[0, 1]$; others are nonnegative counts or money. Non-finite values should be rejected.

Definition 2.4 (Metric specification). A metric specification contains at least a stable name, direction (maximize or minimize), unit or domain, finite-value requirement, and optional tolerance.

```
type MetricSpec struct {
    Name       string
    Direction  Direction
    Unit       string
    Tolerance  float64
}
```

2.3.3 Metrics and constraints are different

A constraint such as “no unauthorized disclosure” should not be encoded as a tiny negative weight in a score. Constraints define admissibility. Metrics compare admissible alternatives.

Definition 2.5 (Constraint observation). A constraint observation is a proposition that must hold under a declared quantifier, such as for every cell, every protected stratum, or with a specified risk bound.

Examples:

- every returned chunk was authorized;
- every score was finite;
- every citation referred to admitted evidence;
- no protected case failed;
- degradation rate did not exceed 1%;
- the lower confidence bound on recall exceeded 0.90.

The quantifier matters. `authorized=true` averaged over cells is meaningless if one cell disclosed protected content.

2.3.4 Missingness is evidence

If a metric is absent because the system failed before producing an answer, that absence should not be converted automatically to zero or omitted from the summary. The observation records the failure status, and the analysis states which metrics are undefined for that status.

Counterexample 2.2 (Complete-case bias). A candidate answers easy cases and fails hard cases. The report computes answer quality only over successful cells. Its conditional mean rises even though user-level success falls. A valid decision must include failure rate and either define a composite outcome or gate completeness before comparing conditional quality.

2.4 Couplings: how two stochastic systems are compared

2.4.1 Marginals do not determine a paired experiment

Let the baseline and candidate outcome kernels for one visible input be

$$K_b: X \rightarrow D(O_b), K_c: X \rightarrow D(O_c).$$

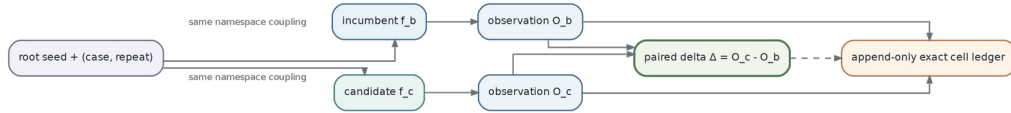
Knowing the two marginal distributions does not determine how samples should be paired. We need a joint process whose marginals are the original systems.

Definition 2.6 (Coupling). A coupling of K_b and K_c is a kernel

$$\Gamma: X \rightarrow D(O_b \otimes O_c)$$

such that discarding either side recovers the other marginal:

$$(id \otimes !) \circ \Gamma = K_b, (! \otimes id) \circ \Gamma = K_c.$$



A paired trial is an explicit coupling whose marginals are the baseline and candidate systems.

Couplings affect variance and interpretation while preserving each system's marginal behavior.

2.4.2 Independent coupling

The simplest coupling samples the two systems independently. If their probabilities are P_b and P_c , the joint is the product

$$\Gamma_{ind}(o_b, o_c) = P_b(o_b) P_c(o_c).$$

This is valid but may be statistically inefficient. Random environmental difficulty can obscure the treatment difference.

2.4.3 Common random numbers

A common-random-number coupling supplies both systems with the same latent disturbance U :

$$U \sim D(U), O_b = f_b(x, U), O_c = f_c(x, U).$$

If baseline and candidate respond similarly to U , the paired difference has lower variance.

The sandbox passes the same root seed to both arms at one case/repeat coordinate. Stable component namespaces determine which stochastic calls are shared.

Worked example 2.1 (Variance reduction). Suppose

$$O_b = \mu_b + U + \epsilon_b, O_c = \mu_c + U + \epsilon_c,$$

with independent zero-mean ϵ_b, ϵ_c and shared disturbance U . The paired difference is

$$O_c - O_b = (\mu_c - \mu_b) + (\epsilon_c - \epsilon_b),$$

so the variance of U cancels. Under independent runs, two independent disturbances would remain in the difference.

2.4.4 Coupling compatibility

Common randomness is meaningful only when calls correspond semantically. A baseline with one reranker call and a candidate with three data-dependent calls cannot simply share “draw 0, draw 1, draw 2” by execution order. The architecture needs stable semantic call labels or a more explicit coupling plugin.

Possible couplings include:

- independent product;
- common random numbers by semantic namespace;
- retained provider-response replay;
- shared workload perturbations;
- antithetic sampling;
- matched random prompts or documents;
- optimal transport couplings in specialized analyses.

Definition 2.7 (Coupling policy). A coupling policy declares how baseline and candidate random sources, provider responses, workload perturbations, or retained artifacts are jointly generated, and which marginal laws it promises.

Applied example. A reranker campaign can declare shared semantic noise for the reranker stage, independent randomness for unrelated answer generation, and replay for an expensive retained embedding response. The policy is identified with the run so the same component can participate in different valid experiments.

2.4.5 The coupling is evidence design, not system identity

A reranker has its own marginal denotation. The decision to compare two rerankers under shared noise belongs to the experiment. This is why root-seed and coupling configuration should live in run identity rather than the component’s standalone semantic identity.

2.5 Exact cell coordinates and durable custody

2.5.1 The cell is the unit of experimental evidence

Definition 2.8 (Experimental cell). A cell is one arm execution and evaluation at an exact coordinate:

$$(run, case, repeat, arm).$$

A paired coordinate contains two cells with the same run, case, and repeat but different arms.

Applied example. In run *fusion-v4*, case *q17*, repeat 3, the baseline cell and candidate cell have distinct arm keys but share the same case material and root coordinate seed. If the process crashes after committing only the baseline, resume executes only the missing candidate cell; it does not create repeat 4.

The coordinate is not a row number or execution order. It is semantic identity. Execution may be parallel, resumed, or reordered while preserving coordinates.

A typical key is:

```
type CellKey struct {
  RunID  string
  CaseID string
  Repeat int
  Arm    ArmID
}
```

2.5.2 Why repeats have identity

A repeat is not “try again until stable.” It is a planned stochastic replicate with a deterministic seed derivation. Repeat 3 of case *q17* means the same coordinate after restart. Adding repeats later should create new coordinates rather than changing old ones.

2.5.3 Completed failure versus missing cell

A failed or cancelled system execution can still produce a completed cell containing status, trace, evaluator observation, and artifacts. A missing cell means no valid durable record exists at that coordinate.

This distinction is essential for resume and exact pairing.

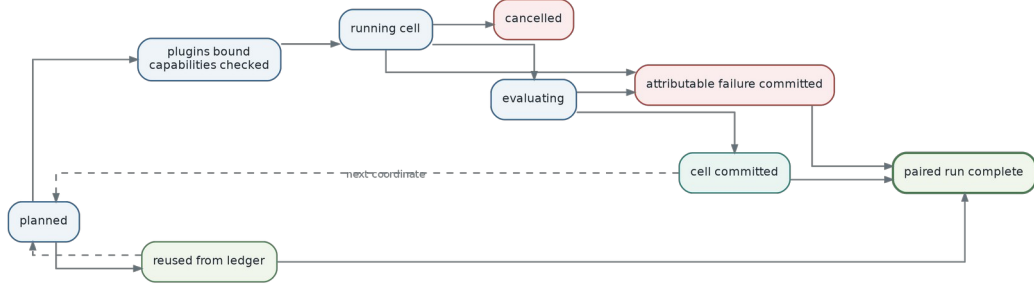
Definition 2.9 (Exact paired coverage). A paired run is complete when every planned case/repeat coordinate has exactly one valid baseline cell and exactly one valid candidate cell. Product failure counts as a cell; absence does not.

2.5.4 Append-only ledger

The sandbox writes cells as JSON Lines. On resume, it validates existing coordinates and executes only missing cells.

The ledger should enforce:

- unique cell keys;
- run and candidate identity compatibility;
- immutable completed cells;
- artifact and trace references;
- atomic append or transactional commit;
- explicit terminal run status;
- no silent overwrite.



One cell transitions through execution, evaluation, validation, and durable commit.

2.5.5 Operational cell rule

A simplified cell program is:

```

if ledger contains valid key:
    return Reused(cell)

outcome = arm.Run(visible(case), seed(key))
observation = evaluator.Evaluate(case, outcome)
cell = validate(key, outcome, observation)
ledger.Append(cell)
return Committed(cell)

```

If `arm.Run` returns a product failure outcome, evaluation still runs if defined. If `ledger.Append` fails, the runner has an interpreter error: it cannot claim the cell is durable.

2.5.6 Resume equivalence

Let P be the ledger prefix before interruption and U the valid suffix after resume. A correct runner should produce the same terminal cell set as uninterrupted execution under the same retained provider responses or stable seeds.

This is a state-machine property, not only a file-format property. Tests should interrupt after every durable boundary.

2.6 Aggregation and sufficient statistics

2.6.1 Raw cells remain authoritative

Aggregation produces a convenient summary, but it discards information. A mean cannot reveal multimodality, missing strata, or one catastrophic failure. Raw cells and native artifacts remain retained.

Definition 2.10 (Mergeable summary). A mergeable summary is a state A with an associative, preferably commutative merge operation such that summaries of disjoint cell sets can be combined without reading the original values again.

For a numeric metric, the state can be

$$(n, s, q, l, u),$$

where n is count, s sum, q squared sum, l minimum, and u maximum. Merge uses addition for n, s, q and min/max for bounds.

Mean and sample variance are derived:

$$\bar{x} = \frac{s}{n}, s^2 = \frac{q - s^2/n}{n - 1}.$$

2.6.2 Aggregation is not statistical inference

A mergeable summary describes observed cells. It does not by itself justify confidence intervals, causal claims, or generalization. Statistical analysis must account for the sampling unit and dependence structure.

If five queries come from the same document family, treating them as independent may understate uncertainty. If repeats share provider artifacts, the effective sample size differs. If a proposer adaptively selected the candidate after seeing development results, naive p-values are optimistic.

2.6.3 Stratification

Aggregate metrics should be available by protected tags, such as:

- authorization scope;
- query intent;
- corpus source role;
- easy/hard classification;
- temporal slice;
- language;
- answer versus abstention cases;
- provider path or fallback class.

A candidate should not hide a severe regression in a small but important stratum behind an aggregate gain.

Worked example 2.2 (Simpson’s paradox). A candidate improves average recall because it is evaluated on more easy cases after failures remove hard cases. Within both easy and hard strata, it is worse. Exact planned cells and failure-complete aggregation prevent the workload composition from changing silently.

2.7 Paired differences and statistical interpretation

2.7.1 Direction-normalized deltas

For metric m , let $d_m = +1$ when larger is better and $d_m = -1$ when smaller is better. The direction-normalized paired difference is

$$\Delta_{i,r,m} = d_m (m_c(i,r) - m_b(i,r)).$$

Positive values always favor the candidate.

The paired mean is

$$\hat{\Delta}_m = \frac{1}{N} \sum_{i,r} \Delta_{i,r,m}.$$

Using paired deltas often reduces variance because case difficulty and shared random disturbance cancel.

2.7.2 Noninferiority versus superiority

Many protected metrics need not improve; they must not regress beyond a tolerance.

Definition 2.11 (Noninferiority). A candidate is noninferior to a baseline on metric m with margin $\epsilon \geq 0$ when the evidence supports

$$E[\Delta_m] \geq -\epsilon.$$

For a maximize metric, this means candidate minus baseline is not worse than $-\epsilon$. For a minimize metric, direction normalization handles the sign.

Definition 2.12 (Target improvement). A candidate satisfies a target-improvement rule on m with minimum $\tau > 0$ when the evidence supports

$$E[\Delta_m] \geq \tau.$$

Worked example 2.3 (Protected recall, target MRR). A product can require recall to be noninferior with margin zero while requiring MRR to improve by at least 0.01. A candidate that preserves recall and improves MRR is promotable; a candidate that improves MRR but loses recall fails the earlier protected rule.

The phrase “supports” depends on the analysis: deterministic exact comparison, lower confidence bound, posterior probability, bootstrap interval, or another declared method.

2.7.3 Adaptive bias

When many candidates are tested and the best observed one is selected, its observed improvement is optimistically biased. A holdout, nested evaluation, sequential correction, or conservative decision policy is needed.

The architecture does not force one statistical method. It ensures that the method receives exact retained cells, pairing, candidate history, and visibility metadata rather than a preselected score.

Fundamentals: confidence interval versus decision. A confidence interval describes an estimator under assumptions. A gate is a policy that maps evidence to an action class. The gate may use a confidence interval, but the interval itself does not decide which regressions are acceptable.

2.8 Decision semantics

2.8.1 Why three verdicts are necessary

Binary pass/fail forces weak evidence into one side. A candidate may satisfy hard safety constraints but lack enough evidence of improvement. Rejecting it says the evidence supports inferiority or inadmissibility; accepting it says the evidence supports promotion. Neither is accurate.

Definition 2.13 (Three-valued decision). The decision set is

$$D = \{Eligible, Undecided, Rejected\}.$$

- **Eligible** means the candidate satisfies every hard rule and the required promotion evidence.
- **Rejected** means at least one hard rule fails.
- **Undecided** means hard rules pass but required positive evidence is absent or a soft rule fails.

Applied example. A candidate with zero security violations but a confidence interval that crosses the target-improvement threshold is **Undecided**. A candidate with one unauthorized disclosure is **Rejected**. A candidate that passes security, noninferiority, budget, and improvement rules is **Eligible**.

2.8.2 Ordered gates

Definition 2.14 (Ordered gate policy). A gate policy is an ordered list of rules. Each rule returns pass/fail, hard/soft classification, and supporting evidence. Evaluation proceeds left to right:

- a failed hard rule yields **Rejected** and may stop;
- a failed soft rule yields at least **Undecided** but later diagnostics may continue;
- all required rules passing yields **Eligible**.

Worked example 2.4 (Why order is visible). Evaluate authorization before an expensive LLM judge. An unauthorized candidate is rejected immediately and its answer text need not be disclosed to the judge. Reversing the order wastes cost and can create another disclosure even though the final verdict would still be rejection.

The order is part of policy identity. Security and integrity are evaluated before relevance, which is evaluated before cost preferences.

```
gate := evidence.Policy{Rules: []evidence.Rule{
    evidence.ConstraintAll{Constraint: "authorized", Hard: true},
    evidence.ConstraintAll{Constraint: "finite_scores", Hard: true},
    evidence.NonInferior{Metric: "recall", Direction: evidence.Maximize, Margin: 0, Hard:
true},
    evidence.NonInferior{Metric: "mrr", Direction: evidence.Maximize, Margin: .03, Hard:
true},
    evidence.Budget{Metric: "latency_ms", Maximum: 1.3, Hard: true},
    evidence.Budget{Metric: "index_units", Maximum: 1200, Hard: true},
    evidence.Improve{Metric: "mrr", Direction: evidence.Maximize, Minimum: .005, Hard:
false},
}}
```

This example is from the companion demo.

2.8.3 Lexicographic safety

A weighted score such as

$$U = 10 \cdot MRR - 0.001 \cdot latency - 1000 \cdot violations$$

still permits one violation to be compensated by enough MRR. If violations are inadmissible, the correct structure is lexicographic or constraint-first:

$$security < integrity < reliability < quality < cost.$$

2.8.4 Budgets

A budget is a hard or soft upper/lower bound on a metric or resource. Examples include latency, cost, index size, remote calls, or carbon budget. A budget can be evaluated per cell, in a protected percentile, or as an aggregate. The quantifier and statistic belong to the rule identity.

2.8.5 Gate evidence

Each rule result should retain:

- rule specification and version;
- metrics/constraints inspected;
- cell coverage and strata;
- threshold or margin;
- statistical method, if any;
- pass/fail result;
- diagnostic reason;
- artifact references.

A final verdict without rule evidence is not auditable.

2.9 Partial orders and Pareto fronts

2.9.1 No universal scalar

Definition (Preorder). A preorder is a relation that is reflexive and transitive. It may leave alternatives incomparable, and two distinct alternatives may be mutually equivalent under the relation. Optimization decisions often use preorders because tolerances and partial information prevent a unique total ranking.

Two eligible candidates may be incomparable. Candidate A has higher MRR but greater latency. Candidate B has lower cost but larger index size. Unless product policy provides an exchange rate, neither dominates.

Definition 2.15 (Pareto dominance). After normalizing metric directions, candidate a weakly dominates b when

$$\forall m, a_m \geq b_m$$

within declared tolerances, and strictly dominates when at least one inequality is strict.

Definition 2.16 (Pareto front). The Pareto front is the set of admissible candidates not strictly dominated by another admissible candidate.

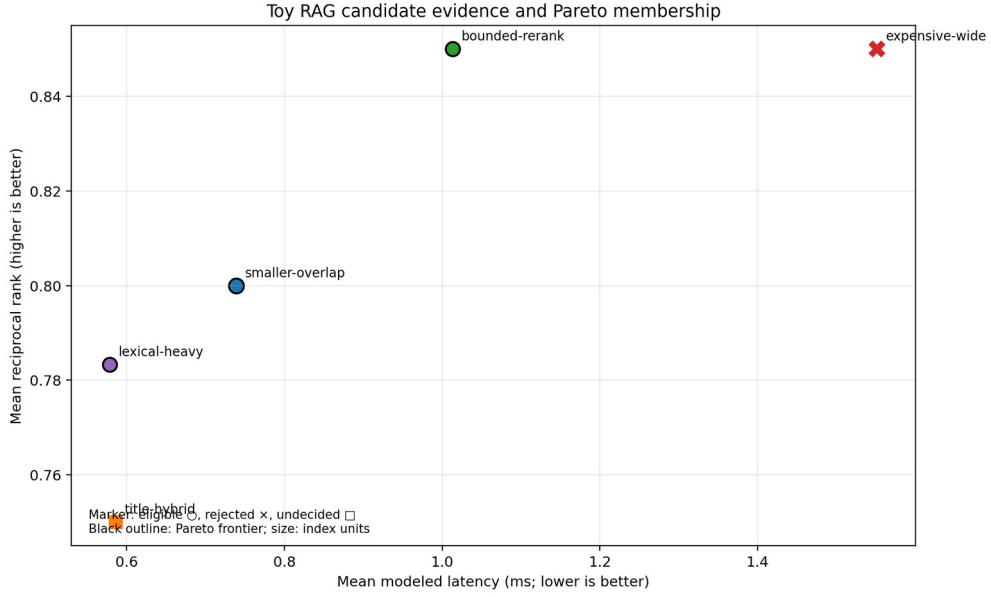
Hard-gate rejection occurs before Pareto comparison. An unsafe candidate is not rescued by performance.

2.9.2 Tolerances and noisy fronts

With stochastic estimates, tiny observed differences should not create artificial dominance. Metric specifications can include practical tolerances or the dominance relation can use confidence/posterior criteria.

2.9.3 Selection remains policy

The Pareto front narrows alternatives; it does not choose one. A selector may prefer maximum MRR, minimum cost, a product utility model, or a human review. Keeping selection separate prevents a hidden scalar objective from contaminating evidence.



Example candidate trade-offs and Pareto membership.

2.10 Adaptive campaigns as stochastic control

2.10.1 History is the state

An adaptive proposer chooses the next candidate based on completed evidence. This makes a campaign a stateful stochastic process.

Definition 2.17 (Campaign history). A campaign history H is an immutable sequence or content-addressed state containing the incumbent, candidates, trial identities, evidence summaries, verdicts, proposer-visible projections, budgets, and stopping information.

Definition 2.18 (Proposer). A proposer is a deterministic or stochastic process

$$\pi: H_{\text{visible}} \rightarrow D(P + \text{Stop})$$

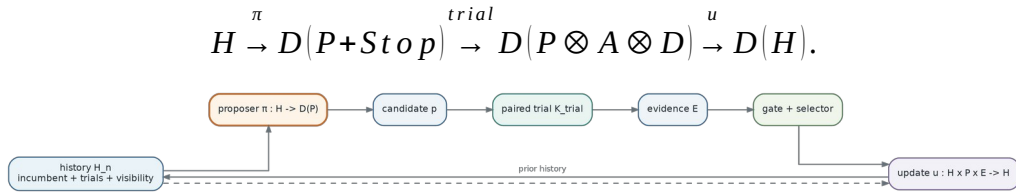
that returns a new candidate parameter or a stop decision from an authorized projection of history.

Definition 2.19 (History update). A history update is

$$u: H \otimes P \otimes A \otimes D \rightarrow H,$$

where A is aggregate evidence and D the verdict.

One campaign step is therefore:



An adaptive campaign is a stochastic state transition over immutable evidence history.

2.10.2 The proposer is not the optimizer field

Grid search, Bayesian optimization, an LLM, a human, and a hand-written enumerator are all proposer implementations. Replacing the proposer should not change:

- system execution semantics;
- evaluator identity;
- coupling policy;
- cell custody;
- gate policy;
- hidden-label visibility.

This is the decisive architectural separation. The “optimizer” is one control plugin, not the owner of the whole experiment.

2.10.3 Visibility classes

A useful campaign distinguishes at least:

1. **system-visible**: query, subject, current release, allowed tools;
2. **evaluator-only**: labels, forbidden evidence, rubric, hidden tests;
3. **proposer-visible**: selected summaries, public diagnostics, budget state;
4. **decision-authority-only**: holdout results, security details, promotion policy.

The exact classes are product-specific, but the information-flow graph should be explicit.

Counterexample 2.3 (Holdout-aware proposer). An LLM proposer receives per-case holdout failures and iteratively writes prompts that solve those exact cases. The final holdout score no longer estimates generalization. The campaign history should expose only development evidence to the proposer and reserve a separate final or sequentially valid evaluation channel.

2.10.4 Stopping

Stopping can occur because:

- the proposer has no candidate;
- budget is exhausted;
- uncertainty is sufficiently low;
- no eligible improvement has appeared for a fixed window;
- the Pareto front has stabilized;
- a human ends the campaign;
- a safety incident suspends evaluation.

Stopping policy belongs to control semantics. It does not retroactively alter completed cells.

2.11 Statistical games and Bayesian lenses: a useful perspective

The separation between forward system and evaluator resembles ideas from statistical games, Bayesian lenses, open games, and categorical cybernetics. These frameworks study systems that have forward behavior and backward context, inference, loss, or adaptation (St. Clere Smithe 2021, 2023; Ghani et al. 2018; Capucci et al. 2022).

The architecture here uses this perspective conservatively. It does not require every evaluator to be a Bayesian inverse or every campaign to be an open game. The useful lesson is structural:

- the forward system does not contain its complete evaluation context;
- evaluation is compositional additional semantics;
- a controller receives a bounded view of evidence and changes parameters;
- local components can retain native evaluators while exposing a common observation boundary.

Fundamentals: lens intuition. An ordinary lens has a forward “get” and a backward “put” that updates a larger structure. A Bayesian lens pairs a forward stochastic channel with a backward inference transformation. In optimization architecture, the analogy is that forward system behavior and backward evaluative/control information are related but should not be collapsed into one opaque method.

2.12 Worked example: comparing two retrieval policies

Consider four cases and one repeat. The baseline and candidate produce reciprocal-rank observations and latency:

Case	Baseline MRR	Candidate MRR	Baseline ms	Candidate ms	Authorized?
q1	1.00	1.00	10	13	yes
q2	0.50	1.00	12	15	yes
q3	1.00	0.50	11	14	yes
q4	0.00	0.50	9	12	yes

The paired MRR deltas are

$$(0, 0.5, -0.5, 0.5),$$

so mean improvement is 0.125. Latency direction is minimize, so direction-normalized deltas are

$$-(3, 3, 3, 3) = (-3, -3, -3, -3).$$

Suppose the policy is:

1. all authorization constraints pass;
2. MRR noninferiority margin is 0.05;
3. mean latency must be at most 14 ms;
4. target MRR improvement is at least 0.10.

Mean candidate latency is 13.5 ms, so the budget passes. The observed mean MRR improvement passes the target. With only four cases, a statistical implementation may still return Undecided because uncertainty is too high. The deterministic descriptive values are not the same as sufficient promotion evidence.

Now suppose q_3 had `Authorized? = no` because the candidate sent one forbidden chunk to a remote reranker. The first hard gate rejects the candidate regardless of its MRR and latency. This is decision semantics, not metric weighting.

2.13 Common design errors

Error 1: evaluator inside the arm

The arm can choose a favorable metric, omit a failure, or inspect hidden labels. Split system and evaluator interfaces.

Error 2: no explicit coupling

Passing the same integer seed to two arbitrary programs does not guarantee comparable random calls. Declare semantic namespaces and coupling compatibility.

Error 3: average over whichever cells exist

Missing candidate cells change workload composition. Require exact paired coverage before comparison.

Error 4: one scalar objective

A weighted score hides inadmissibility and arbitrary exchange rates. Use ordered hard gates and a partial order among survivors.

Error 5: the proposer sees everything

Adaptive proposal overfits evaluation details. Give it a versioned, bounded projection of history.

Error 6: summaries replace raw evidence

A mean cannot diagnose outliers, strata, or failures. Retain cells and native artifacts.

2.14 Chapter summary

An experiment is itself a typed program. A case contains visible input and hidden evaluator material. The evaluator is a separate product-owned process. A coupling specifies how baseline and candidate stochastic outcomes are jointly sampled. Exact case/repeat/arm coordinates and append-only custody make pairing and resume well defined.

Observations contain metrics, constraints, statuses, traces, and native artifacts. Mergeable summaries support scalable reporting but do not replace raw cells or statistical analysis. Decision semantics uses three-valued verdicts, ordered hard and soft gates, noninferiority, target improvement, and budgets. Pareto fronts preserve genuine trade-offs. An adaptive proposer is a control process over a bounded view of immutable campaign history; it does not own the system or evidence semantics.

Chapter 3 turns these distinctions into a small core and a plugin architecture.

2.15 Exercises

Conceptual exercises

1. **Visibility split.** Design a case type for answer-quality evaluation. Mark every field as system-visible, evaluator-only, proposer-visible, or decision-authority-only.
2. **Evaluator boundary.** Give an example of a validator that properly belongs inside system semantics and a judge that must remain evaluator-owned.
3. **Missingness.** Explain the difference between a failed cell and a missing cell. How should each affect paired coverage?
4. **Metric direction.** Define specifications for recall, latency, cost, violation count, and answer length. Which are metrics, constraints, or both under different policies?
5. **Three verdicts.** Construct a scenario for each of `Eligible`, `Undecided`, and `Rejected`.
6. **Pareto front.** Given candidates $(quality, cost)$: $A=(0.8,10)$, $B=(0.9,15)$, $C=(0.85,9)$, $D=(0.7,20)$, identify the Pareto front when quality is maximized and cost minimized.
7. **Visibility leak.** Describe how a proposer can overfit even when the system never sees labels.

Mathematical exercises

8. **Coupling marginals.** For Bernoulli distributions with probabilities p_b and p_c , construct the independent coupling and verify both marginals.
9. **Maximal positive coupling.** For Bernoulli variables, construct a coupling that maximizes the probability they are equal. Compare its variance for the paired difference with independent sampling.
10. **Paired variance.** Derive

$$\text{Var}(O_c - O_b) = \text{Var}(O_c) + \text{Var}(O_b) - 2\text{Cov}(O_c, O_b).$$

Explain why positive covariance can help.

11. **Summary monoid.** Prove that merging states (n, s, q, l, u) is associative and commutative.
12. **Noninferiority signs.** Write the noninferiority inequality for a minimize metric without using direction normalization, then show the normalized form.
13. **Dominance preorder.** Prove that weak Pareto dominance is reflexive and transitive. Why is it not antisymmetric when tolerances are used?
14. **Gate noncommutativity.** Give two gate rules whose order changes the diagnostic output or computational cost. Explain why policy order belongs to identity.

Programming exercises

15. **Case projection.** Implement separate `VisibleCase` and `EvaluationCase` types so the arm cannot access labels.
16. **Coupled runner.** Implement paired execution with a root seed derived from run, case, and repeat. Add configurable independent and shared-namespace couplings.
17. **JSONL ledger.** Build an append-only cell ledger with duplicate-key rejection and resume. Inject a crash after append and verify recovery.
18. **Complete-pair check.** Write a validator that rejects duplicate, unmatched, or missing baseline/candidate coordinates.
19. **Summary aggregation.** Implement mergeable metric summaries and stratification by tags.
20. **Gate engine.** Implement ordered hard and soft rules with evidence-rich results and three-valued verdicts.
21. **Pareto front.** Implement tolerance-aware Pareto filtering. Test equal and incomparable candidates.
22. **History projection.** Define a proposer-visible history schema that excludes hidden holdout details and native sensitive artifacts.

Applied RAG exercises

23. Design a paired experiment for lexical/vector fusion weights. Which artifacts can be reused, and what coupling is appropriate?
24. Design an experiment comparing a local deterministic reranker with a remote stochastic reranker. Include disclosure, failure, latency, and quality gates.
25. Design a multi-turn agent evaluation where the experimental unit is a conversation rather than a query. What is one cell?
26. Define protected strata for an administrative RAG system with access scopes and source roles.
27. Construct a counterexample where retrieval MRR improves but answer quality or frontend usefulness regresses.

Research exercises

28. Compare frequentist confidence intervals, Bayesian posterior decision rules, and sequential tests as implementations of “supports noninferiority.”
29. Read about comparison of statistical experiments in categorical probability. How does informativeness differ from the paired coupling used here?
30. Model the campaign as a Markov decision process. Which state variables are legitimate for the proposer to observe?

Chapter 3. A Small Semantic Core and a Strong Plugin Architecture

Learning objectives

By the end of this chapter, you should be able to:

1. state a minimality criterion for a shared optimization core;
2. separate a plugin's pure semantic specification from its effectful runtime binding;
3. design typed factory interfaces without turning a catalog into a service locator;
4. declare effects and capabilities so invalid compositions can be rejected before execution;
5. attach executable law certificates to plugin specifications;
6. distinguish compile-time plugins from dynamic out-of-process plugins;
7. use exact and sampled interpreters for the same semantic signature;
8. explain how package boundaries encode the four semantic layers;
9. trace one experiment cell through binding, execution, evaluation, validation, and commit;
10. identify common plugin designs that are flexible syntactically but weak semantically.

3.1 The architectural problem: extensibility without semantic collapse

3.1.1 Why plugin systems become frameworks

A RAG optimization platform wants extensibility. Teams want to add chunkers, embedding models, indexes, query rewriters, rerankers, judges, proposers, and selectors. The tempting answer is one registry of named implementations and one universal request/response envelope.

That design often grows into a framework with three pathologies:

1. **Type erasure.** Every plugin receives `map[string]any`, JSON, or a generic context object.
2. **Hidden authority.** A plugin can read stores, labels, credentials, and campaign state that are unrelated to its declared role.
3. **Semantic drift.** Names remain stable while prompts, model aliases, policies, or fallback behavior change.

The opposite extreme is a monolith where every product case is hard-coded into the core. That preserves local types but destroys reuse and independent evolution.

Motivation. The core must be small enough to understand and prove laws about, yet strong enough that domain plugins cannot erase the distinctions established in Chapters 1 and 2.

3.1.2 Minimality criterion

Definition 3.1 (Core minimality criterion). A concept belongs in the shared semantic core only when at least one of the following is true:

- it is required to state composition laws;
- it prevents invalid comparison across product domains;
- both exact and sampled interpreters need it;
- at least two independent domain grafts require it;
- it defines identity or custody shared by all campaigns.

By this criterion, the core includes typed processes, specifications, stable seeds, outcomes, traces, cell coordinates, evidence summaries, gate order, and campaign history. It does not include chunk semantics, SQL schemas, source roles, judge rubrics, product widgets, or deployment policy.

Worked example 3.1 (Should chunk be in the core?). A chunk is central to RAG but meaningless for image compression, database query optimization, or compiler tuning. It fails the cross-domain criterion. `ragkit` or a RAG adapter owns `Chunk`; the generic process core sees only typed input/output ports.

3.2 Four plugin families

The four semantic layers imply four plugin families.

Definition 3.2 (System plugin). A system plugin realizes a typed deterministic or stochastic process. Examples: chunker, embedder, reranker, query interpreter.

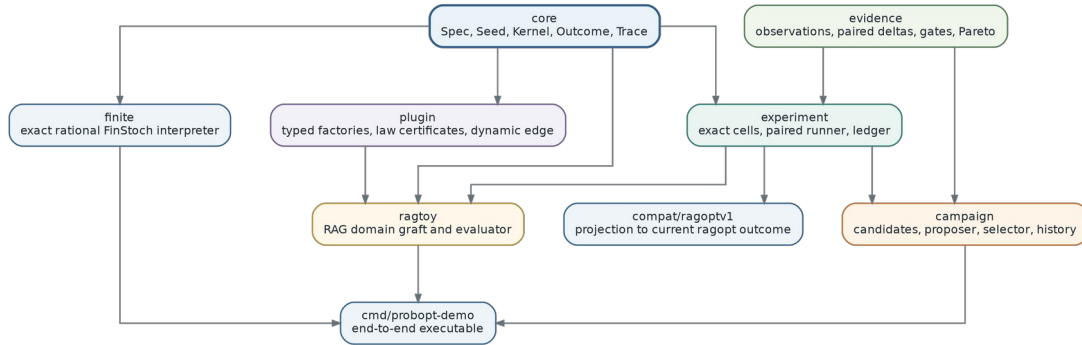
Definition 3.3 (Evaluator plugin). An evaluator plugin maps a hidden case and instrumented outcome to product-owned observations and native diagnostic artifacts.

Definition 3.4 (Decision plugin). A decision plugin implements an aggregation, risk analysis, ordered gate, Pareto relation, or selector over retained evidence.

Definition 3.5 (Control plugin). A control plugin proposes candidates or stopping decisions from an authorized projection of campaign history.

These families should not be unified into one interface. A component allowed to run the system, inspect labels, change the gate, and select itself defeats experimental attribution.

A compact architectural diagram is shown below.



Typed plugin families attach to a small semantic core through explicit ports.

3.3 Pure specification and effectful binding

3.3.1 The split

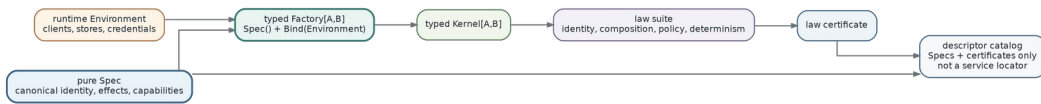
A plugin has two lives. Before execution, it is a pure description that can be inspected, hashed, compared, authorized, and placed in a process graph. During execution, it needs clients, credentials, file handles, stores, caches, and worker pools.

Definition 3.6 (Plugin specification). A plugin specification is a pure, canonical, content-identified description of the component's claimed semantics. It contains semantic configuration, version, effects, capabilities, schemas, and stable random namespace.

Definition 3.7 (Runtime binding). Runtime binding resolves a specification against an environment to produce an executable typed process. Binding may fail when required capabilities are unavailable. It may not silently alter the specification.

The preferred interface is:

```
type Factory[A, B any] interface {
  Spec() core.Spec
  Bind(
    context.Context,
    plugin.Environment,
  ) (core.Kernel[A, B], error)
}
```



A typed factory separates pure specification from operational binding.

3.3.2 Why a factory rather than a singleton

A singleton plugin instance often captures process-local state at construction time. It may hold a mutable RNG, client, cache, or endpoint. Its meaning cannot be inspected without running it, and tests cannot bind the same semantics to a reference environment.

A factory supports:

- preflight validation before expensive resources are created;
- exact semantic identity independent of process location;
- separate reference and production bindings;
- dependency-closure planning from specifications;
- capability and effect policy;
- fresh or shared runtime instances according to lifecycle;
- explicit failure when the environment cannot realize the claim.

3.3.3 The environment is not the specification

The sandbox environment is deliberately small:

```
type Environment interface {
    Lookup(name string) (any, bool)
}
```

A production environment should be more typed than this example, perhaps with capability-specific resolvers. The semantic distinction is the important part.

Fields that usually belong in the **specification**:

- model and provider revision when it changes behavior;
- prompt or tool-description content identity;
- chunk size, fusion weights, rerank depth;
- output schema and fallback policy;
- random namespace;
- disclosure class and required data policy;
- deterministic versus stochastic declaration.

Fields that usually belong in the **environment**:

- credentials;
- sockets and process-local clients;
- worker pool size;
- temporary directories;
- tracing exporters;
- immutable artifact locator for an already identified artifact;
- secret material that does not alter semantics.

The classification is not universal. If an endpoint aliases different models by region, endpoint selection may alter denotation and must be represented semantically.

Counterexample 3.1 (Mutable alias). A spec says `model="best-reranker"` while the environment resolves that alias to whichever deployment is current. The same spec ID can produce different rankings. Either resolve the alias to an immutable revision in the spec or classify the process as observationally versioned and retain material responses.

3.4 Canonical specification identity

3.4.1 The spec value

The sandbox specification is:

```
type Spec struct {
    Kind      string
    Name      string
    Version   string
    Config     map[string]any
    Effects    []Effect
    Capabilities []string
    RandomNamespace string
    ID        string
}
```

`NewSpec` validates required fields, sorts effects and capabilities, canonically encodes the content, and computes a domain-separated SHA-256 identifier.

```
sum := sha256.Sum256(
    append([]byte("probopt-spec/v1\x00"), canonicalBytes...),
)
```

Definition 3.8 (Semantic identity). Semantic identity is a content-derived identifier over every declared input capable of changing the denotation or protected trace of a component, excluding fields declared purely operational.

3.4.2 Identity is a commitment, not a proof

A specification ID says: “this implementation claims these semantics.” It does not prove the code is correct, the provider honored its version, or the plugin satisfies its laws. Law certificates, runtime validation, artifact verification, and differential tests provide evidence for the claim.

3.4.3 Domain separation

Hashes are domain-separated so identical bytes in different semantic roles do not collide conceptually:

```
probopt-spec/v1 || canonical_spec
probopt-seed/v1 || root_material
```

```
probopt-split/v1 || parent_seed || labels
candidate/v1    || parameter_value || locks
```

Domain separation prevents one digest from being misinterpreted as another object type and makes schema evolution explicit.

3.4.4 Canonicalization obligations

Canonical encoding should define:

- map-key order;
- numeric representation and finite-value rules;
- treatment of omitted versus zero fields;
- list order semantics;
- Unicode normalization if text is identity-bearing;
- schema version;
- behavior for unknown fields;
- content references versus paths.

An unstable encoder makes cache and campaign identity nondeterministic.

Worked example 3.2 (Order-insensitive capabilities). The capability lists {"vector", "filter"} and {"filter", "vector"} describe the same set. Sorting them before hashing preserves identity. A stage list such as [rewrite, search, rerank] is ordered and must not be sorted.

3.5 Typed kernels and composition combinators

The sampled process interface is:

```
type Kernel[A, B any] interface {
  Spec() Spec
  Run(context.Context, Request[A]) (Outcome[B], error)
}
```

The core supplies:

```
func Identity[A any](name string) Kernel[A, A]
func Lift[A, B any](spec Spec, f func(A) (B, error)) Kernel[A, B]
func Compose[A, B, C any](f Kernel[A, B], g Kernel[B, C]) Kernel[A, C]
func Tensor[A, B, C, D any](f Kernel[A, B], g Kernel[C, D])
  Kernel[Pair[A, C], Pair[B, D]]
func Fanout[A, B, C any](f Kernel[A, B], g Kernel[A, C])
  Kernel[A, Pair[B, C]]
```

3.5.1 Why combinators belong in the core

These operations are domain-independent and correspond to the process laws. They ensure that specifications, traces, resources, failures, and stable seeds compose consistently. Product code should not reimplement slightly different sequential semantics in every RAG stage.

3.5.2 Why not expose unrestricted bind?

A fully general monadic bind chooses the next computation from a runtime value. It is expressive but hides the static process graph. For optimization, preflight effect checks, dependency closure, and artifact planning benefit from an inspectable structure. The core therefore emphasizes Compose, Tensor, and typed builders. Dynamic data-dependent control is still possible inside explicit agent or branching plugins whose semantics and trace declare it.

This is analogous to the motivation for Arrows and applicative structure in functional programming (Hughes 2000; McBride and Paterson 2008).

3.6 Effects and capabilities

3.6.1 Effects describe what a component may do

The sandbox declares coarse effects:

```
const (
  EffectPure      Effect = "pure"
  EffectRandom    Effect = "random"
  EffectLocalIO   Effect = "local_io"
  EffectRemoteIO  Effect = "remote_io"
  EffectStateful  Effect = "stateful"
  EffectDisclosure Effect = "disclosure"
)
```

Definition 3.9 (Effect declaration). An effect declaration is a conservative statement of externally relevant actions a component may perform. Effects participate in semantic identity and policy validation.

A remote reranker may declare `random`, `remote_io`, and `disclosure`. A deterministic local chunker declares `pure`. A cell ledger declares `local_io` or `stateful`, but it is an interpreter component rather than part of the optimized system’s denotation.

3.6.2 Capabilities describe what the component provides or requires

Definition 3.10 (Capability). A capability is a named semantic or operational property used during composition and binding, such as `filter_pushdown`, `exact_scores`, `streaming`, `deterministic_build`, or `authorization_certificate`.

Capabilities should be versioned or schema-qualified when their meaning is not obvious. A Boolean string is weaker than a typed capability record, but it is better than discovering incompatibility after execution.

3.6.3 Effect policy

The sandbox policy validates allowed effects and required capabilities:

```
type EffectPolicy struct {
    Allowed          map[core.Effect] bool
    RequiredCapabilities map[string] bool
}
```

A production graph validator can enforce rules such as:

- no `disclosure` stage before authorization certificate;
- no remote provider for restricted data class;
- deterministic evaluator required for a particular certification suite;
- index backend must support filter pushdown;
- agent tool must support idempotent call IDs;
- dynamic plugin must run out of process.

3.6.4 Effects compose

The composite effect set is the union of leaf effects, but policy may depend on order and data flow. The set `{authorization, disclosure}` alone does not prove authorization occurs first. The process graph and trace obligation are also needed.

Design consequence. Use effects for fast conservative preflight and a typed graph law for ordering-sensitive properties. Do not expect a flat tag set to prove information-flow safety.

3.7 Random namespaces and trace obligations as plugin contracts

A stochastic plugin must declare a stable random namespace. Its law suite should test:

- repeated execution at the same semantic coordinate is reproducible under the reference pseudorandom interpreter;
- distinct semantic calls use distinct labels;
- parallel scheduling does not alter leaf draws;
- baseline/candidate namespace sharing behaves according to the coupling policy;
- retries either reuse or derive attempts according to declared semantics.

A plugin’s trace obligations can include:

- leaf spec ID;
- model/provider revision;
- input and output artifact references;
- fallback class;
- disclosure certificate;
- retry attempt and retained response;
- resource measurement source;
- product status classification.

Trace law tests should compare normalized semantic traces, not timestamps.

3.8 Law certificates and substitutability

3.8.1 Why interfaces are insufficient

Two chunkers can implement the same Go interface while differing in determinism, source-lineage preservation, or local-change stability. Two vector indexes can implement `Search` while one returns deleted items or non-finite scores. A replacement is safe only relative to laws.

Definition 3.11 (Plugin law). A plugin law is an executable or formal obligation over a component’s behavior, such as determinism, ordering, lineage preservation, authorization, boundedness, or refinement to a reference interpreter.

Definition 3.12 (Law certificate). A law certificate is retained evidence that a particular specification and implementation build passed a named suite under identified fixtures and environment. It is evidence, not a proof object.

The sandbox API is intentionally small:

```
type Law interface {
    Name() string
    Check(context.Context) error
}

func Certify(
    ctx context.Context,
    spec core.Spec,
    laws ...Law,
) Certificate
```

3.8.2 Domain law examples

A deterministic chunker might satisfy:

- repeated output equality;
- every chunk range lies within the source document;
- chunk text equals the source slice;
- stable total ordering;
- changes in document A do not alter chunks of document B.

A stochastic reranker might satisfy:

- finite scores;
- output is a permutation/subset of input candidate IDs;
- output count is bounded;
- authorization certificate precedes remote disclosure;
- same seed namespace produces pathwise reproducibility;
- timeout produces declared fallback status and trace.

An evaluator might satisfy:

- every completed cell yields a total observation schema;
- metrics are finite or explicitly missing;
- hard constraints fail closed;
- hidden labels never enter the system input artifact;
- native artifact references resolve.

3.8.3 Certificate invalidation

A certificate is bound to:

- spec ID;
- implementation artifact/build ID;
- law-suite ID and version;
- fixtures or generators;
- interpreter/environment class;
- execution time and toolchain where relevant.

Changing any of these can invalidate the certificate. A catalog must not attach an old certificate to a new binary merely because the human-readable name matches.

3.8.4 Substitutability is relation-specific

Definition 3.13 (Substitutability). A plugin p' is substitutable for p in a context when it has compatible typed ports and capabilities and satisfies the context's required laws or refinement relation.

There is no context-free universal substitutability. A faster approximate vector index may substitute for an exact index in a serving context with a recall tolerance but not in the exact oracle used to certify other ANN indexes.

Applied example. A deterministic local reranker may replace a remote reranker in a restricted-data route if it preserves the required ranking noninferiority and removes the disclosure effect. The same replacement may be invalid in a campaign whose hypothesis specifically studies the remote model's semantic behavior.

3.9 Catalogs are not service locators

3.9.1 Descriptor custody

The sandbox catalog stores specifications and certificates:

```
type Catalog struct {
    specs map[string]core.Spec
    certs map[string]Certificate
}
```

It cannot return an executable any by name. Normal Go wiring retains typed ports.

Definition 3.14 (Descriptor catalog). A descriptor catalog is a registry of pure specifications, schemas, effects, capabilities, documentation, implementation identities, and law certificates. It supports inspection and selection but not untyped invocation.

Applied example. An admin UI can list all certified reranker specs that provide `authorization_certificate` and show their model revision and effects. Product compilation then chooses a typed factory. The UI never receives a generic executable handle.

3.9.2 Why service locators are dangerous

A service locator encourages code like:

```
x := catalog.Resolve("reranker")
result := x.(interface {
    Run(context.Context, any) (any, error)
}).Run(ctx, payload)
```

The compiler no longer checks ports. Runtime configuration decides whether an object is a reranker, evaluator, or proposer. Hidden dependency lookup makes effect and information-flow analysis incomplete.

Compile-time plugin sets can still be configurable. A product may choose among typed factories using a generated switch or constructor:

```
func BuildReranker(spec RerankerChoice) (Factory[Input, Ranking], error) {
    switch spec.Kind {
    case "local_cross_encoder":
        return NewLocalCrossEncoder(spec.Config), nil
    case "remote_provider":
        return NewRemoteReranker(spec.Config), nil
    default:
        return nil, fmt.Errorf("unknown reranker kind %q", spec.Kind)
    }
}
```

The selection is dynamic; the returned port remains typed.

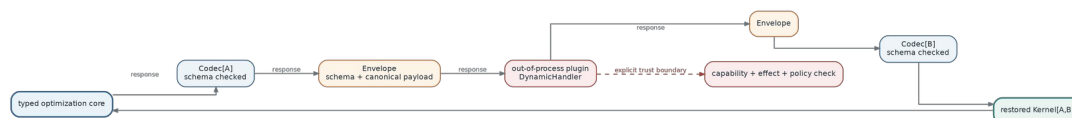
3.9.3 Generated registries

For many plugin kinds, code generation can produce typed registries from descriptor files. Generation preserves discoverability without reflection-heavy runtime erasure. The generated code can also validate schema versions and emit documentation.

3.10 Dynamic plugins at explicit edges

3.10.1 When dynamic loading is necessary

Out-of-process workers, independently deployed providers, user-supplied evaluators, or language-agnostic plugins require dynamic schemas. The solution is not to make the entire core dynamic. It is to isolate the dynamic boundary.



Dynamic plugins cross an explicit schema and capability boundary and are adapted back to typed kernels.

Definition 3.15 (Dynamic plugin edge). A dynamic plugin edge is an explicit transport boundary with versioned input/output schemas, capability grant, effect policy, codec, trace contract, and failure classification. It is immediately adapted to a typed internal process.

Applied example. A Python answer judge runs in a separate worker. The Go campaign sends an `answer-eval-request/v2` envelope containing only the hidden case projection and artifact references, grants no build or deployment capability, validates an `answer-observation/v3` response, and converts it immediately to `Evaluator[Case, Answer]`.

The sandbox defines:

```
type Envelope struct {
    Schema string
    Payload json.RawMessage
}

type DynamicHandler interface {
```

```

Spec() core.Spec
InputSchema() string
OutputSchema() string
Handle(
    context.Context,
    core.Seed,
    Envelope,
) (Envelope, core.Trace, error)
}

```

`AdaptDynamic[A, B]` checks the schemas, encodes typed input, calls the handler, validates returned schema, decodes typed output, and produces `core.Kernel[A, B]`.

3.10.2 Dynamic metadata requirements

A production dynamic descriptor should include:

- immutable plugin and implementation ID;
- input/output schema IDs;
- semantic spec and random namespace;
- allowed effects and required capabilities;
- resource limits;
- transport and authentication policy;
- data-disclosure policy;
- law certificates;
- timeout, retry, and idempotency semantics;
- trace schema;
- lifecycle and compatibility version.

3.10.3 Transport failure versus product failure

A network disconnect before a valid response may be an interpreter failure or a declared provider failure depending on the process contract. Product failure must be represented in the declared output schema. Otherwise the experiment cannot distinguish a candidate’s behavior from broken infrastructure.

3.10.4 Trust boundary

Dynamic plugins should receive least authority. A reranker receives only authorized candidate text and a request-scoped capability, not the full case, campaign history, or artifact store. An evaluator receives hidden labels but cannot mutate the system release. A proposer receives summaries but not holdout artifacts.

3.11 Exact and sampled interpreters

3.11.1 One signature, multiple realizations

A semantic operation can have:

- an exact finite interpreter;
- a deterministic local reference interpreter;
- a sampled local interpreter;
- a production concurrent/batched interpreter;
- a remote dynamic interpreter;
- a replay interpreter over retained artifacts.

The specification and typed port remain stable. Each interpreter makes different operational trade-offs.

3.11.2 Why both exact and sampled models are necessary

The exact model decides small algebraic laws. The sampled model represents effects and real providers. The reference model supports differential testing. The production model supplies performance. A replay model makes incidents and paired experiments reproducible.

The architecture should avoid the false choice between “pure theory” and “real runtime.” The small exact model defines the contract that the runtime refines.

3.11.3 Interpreter refinement examples

- A batched embedder refines the pointwise embedder when output vectors, failure attribution, and trace lineage agree under the declared model version.
- A cached deterministic chunker refines uncached execution when cache hits verify exact output identity and do not suppress required trace facts.
- An ANN searcher refines exact vector search only under a workload-relative recall/rank relation, not exact equality.
- A concurrent lexical/vector interpreter refines sequential tensor when semantic trace is schedule-independent and resource measurement follows its declared algebra.

3.12 Package architecture

The companion module is divided according to semantic ownership:

probopt/	
core/	typed sampled processes, specs, seeds, outcomes, traces
finite/	exact rational stochastic kernels and laws
plugin/	factories, policies, certificates, catalogs, dynamic edges
experiment/	cases, evaluators, cells, ledgers, paired execution
evidence/	metrics, summaries, deltas, gates, Pareto
campaign/	candidates, builders, proposers, selectors, history
ragtoy/	RAG-specific domain graft
compat/	boundary projection to existing ragopt contracts
cmd/	executable demonstration

Dependency direction matters:

- `core` does not import RAG, evaluators, campaigns, or plugins;
- `finite` is an independent exact model;
- `plugin` depends on `core` but not on product domains;
- `experiment` depends on `core` and `evidence`;
- `campaign` depends on `experiment/evidence` but not on RAG;
- `ragtoy` imports the generic packages and supplies domain meaning.

This keeps the core small while making the RAG graft strong rather than ad hoc.

3.13 Operational semantics of a cell and campaign

3.13.1 Cell states

A cell can move through:

```
Planned
-> Bound
-> Executing
-> OutcomeProduced
-> Evaluated
-> Validated
-> Committed
```

Alternative terminals include reused, cancelled, product-failed-but-committed, and interpreter-failed. Only a committed or validated reused cell satisfies a coordinate.

3.13.2 Binding rule

Given factory F with spec s and environment ρ :

1. validate effect/capability policy against $\mathcal{S}$;
2. call `Bind`;
3. reject nil kernel;
4. verify `kernel.Spec().ID == s.ID`;
5. retain binding environment class in operational metadata, not semantic config unless behaviorally relevant.

3.13.3 Campaign state

Campaign history includes the initial and current incumbent plus immutable trials:

```
type History[P any] struct {
  Initial  Candidate[P]
  Incumbent Candidate[P]
  Trials   []Trial[P]
}
```

A campaign round:

```
propose unseen candidates
for each candidate:
  build typed kernel
  run paired experiment
  aggregate evidence
  apply ordered gate
compute admissible/Pareto set
selector may advance incumbent
append immutable round history
```

The proposer and selector are separate. A proposer generates hypotheses; a selector applies product policy among eligible evidence.

3.14 Worked example: a typed remote reranker plugin

3.14.1 Domain port

```

type RerankInput struct {
    Query      string
    Candidates []AuthorizedCandidate
    Certificate AuthorizationCertificate
}

type Ranking struct {
    IDs    []string
    Scores []float64
}

```

The input type makes authorization a prerequisite. The reranker cannot be called with arbitrary unfiltered chunks.

3.14.2 Specification

```

spec := core.MustSpec(
    "reranker",
    "remote-cross-encoder",
    "v2",
    map[string]any{
        "model_revision": "ce-2026-07-14",
        "max_candidates": 40,
        "fallback":       "use_fused_order",
        "input_schema":   "rerank-input/v3",
        "output_schema":  "ranking/v2",
    },
    []core.Effect{
        core.EffectRandom,
        core.EffectRemoteIO,
        core.EffectDisclosure,
    },
    []string{
        "authorization_certificate",
        "bounded_output",
    },
    "retrieval/rerank/remote-cross-encoder",
)

```

3.14.3 Factory

```

type RemoteFactory struct {
    spec core.Spec
}

func (f RemoteFactory) Spec() core.Spec { return f.spec }

func (f RemoteFactory) Bind(
    ctx context.Context,
    env plugin.Environment,
) (core.Kernel[RerankInput, Ranking], error) {
    raw, ok := env.Lookup("reranker-client")
    if !ok {
        return nil, errors.New("reranker client unavailable")
    }
    client, ok := raw.(RerankerClient)
    if !ok {
        return nil, errors.New("wrong reranker client type")
    }

    return core.KernelFunc[RerankInput, Ranking]{
        S: f.spec,
        F: func(
            ctx context.Context,
            req core.Request[RerankInput],
        ) (core.Outcome[Ranking], error) {
            if err := verify(req.Input.Certificate, req.Input.Candidates); err != nil {
                return core.Failed[Ranking](
                    "authorization",
                    err.Error(),
                    core.Event("authorization_rejected", f.spec.ID, nil),
                ), nil
            }
        }
    }
}

```

```

localSeed := req.Seed.For(f.spec, "provider-call")
ranking, usage, err := client.Rerank(
    ctx,
    req.Input,
    localSeed,
)
if isTimeout(err) {
    fallback := fusedOrder(req.Input.Candidates)
    out := core.Degraded(
        fallback,
        "reranker_timeout",
        core.Event("reranker_fallback", f.spec.ID, nil),
    )
    out.Resources = usage
    return out, nil
}
if err != nil {
    return core.Outcome[Ranking]({}, err)
}
if err := validateRanking(ranking, req.Input.Candidates); err != nil {
    return core.Failed[Ranking](
        "invalid_ranking",
        err.Error(),
        core.Event("invalid_output", f.spec.ID, nil),
    ), nil
}
out := core.Success(
    ranking,
    core.Event("reranked", f.spec.ID, map[string]string{
        "certificate": req.Input.Certificate.ID,
    })),
)
out.Resources = usage
return out, nil
}, nil
}

```

This example illustrates the doctrine:

- authorization is in the typed input;
- semantic provider revision and fallback are in the spec;
- the client and credentials come from binding;
- random draw is namespace-derived;
- timeout is degraded product behavior;
- protocol corruption is interpreter error or invalid product output according to the contract;
- trace retains certificate lineage;
- output laws are validated.

3.14.4 Law suite

A certificate should test:

1. unauthorized marker text never reaches the client spy;
2. output IDs are a unique subset/permutation of input IDs;
3. every score is finite;
4. output length is bounded;
5. same coordinate seed produces the same mock stochastic response;
6. timeout returns fused order with degraded status;
7. spec ID changes when model revision, fallback, or schema changes;
8. worker count and endpoint credential rotation do not change spec ID when semantics remain fixed.

3.15 Common plugin anti-patterns

Anti-pattern 1: registry of executables by string

It erases types and hides dependencies. Store descriptors and certificates; wire typed factories.

Anti-pattern 2: `spec()` reads environment variables

The same process can obtain a different identity depending on host state. Construct specs from explicit canonical configuration before binding.

Anti-pattern 3: `bind` silently changes the spec

A factory requested model revision A but only revision B is available, so it binds B and keeps A's ID. This is unsound. Binding must fail or the caller must construct a new spec.

Anti-pattern 4: certificate by plugin name

A certificate for cross-encoder is reused after code or model changes. Bind certificates to spec, implementation build, suite, fixtures, and environment class.

Anti-pattern 5: dynamic everywhere

JSON envelopes inside the core make every composition runtime-checked. Keep dynamic transport at explicit edges and recover typed ports immediately.

Anti-pattern 6: effects are documentation only

If effect declarations are never validated, a supposedly local evaluator can make remote calls or a proposer can read hidden artifacts. Enforce effect/capability policy at graph validation and binding.

Anti-pattern 7: plugin owns its cache key

A buggy or self-serving plugin can omit behaviorally material inputs. Cache identity should be derived from canonical process spec and typed input by shared infrastructure, with domain-specific additions verified by laws.

3.16 Chapter summary

A strong plugin architecture does not maximize the number of things that can be loaded by name. It preserves typed ports and semantic roles while allowing independent implementations to bind at runtime.

The core contains only process composition, canonical specifications, stable randomness, instrumented outcomes, traces, evidence custody, decisions, and control state. Plugins are divided into system, evaluator, decision, and control families. A typed factory separates pure `Spec()` from effectful `Bind(Environment)`. Effects and capabilities permit preflight policy. Law certificates provide version-bound evidence of substitutability. Descriptor catalogs support discovery without becoming service locators. Dynamic plugins are confined to explicit schema-checked edges and adapted back to typed kernels.

Exact and sampled interpreters realize the same semantic signatures at different fidelity. The result is a small core with strong semantics onto which RAG and other optimization domains can be grafted.

3.17 Exercises

Conceptual exercises

1. Apply the core minimality criterion to these concepts: `Prompt`, `CandidateID`, `Trace`, `AccessScope`, `ParetoFront`, `VectorDimension`, `CellKey`, `Widget`.
2. Classify ten components in your system as system, evaluator, decision, or control plugins. Identify any component that currently spans families.
3. For a local embedding model, list fields that belong in `Spec` and fields that belong in `Environment`.
4. Explain why a spec ID is a commitment rather than proof.
5. Give an example where a capability set is insufficient and an ordering law is required.
6. Describe a legitimate use of a dynamic plugin edge and an illegitimate use inside the typed core.
7. State a context in which an approximate component is substitutable and another in which it is not.

API design exercises

8. Design a typed `Factory[Document, []Chunk]` interface with a domain-specific `ChunkerSpec` that compiles to `core.Spec`.
9. Replace `Environment.Lookup(string)` any with typed capability resolvers. Keep the factory generic over input/output types.
10. Define a specification schema for a prompt-backed LLM judge. Which fields must change identity?
11. Define effects and capabilities for a local SQLite vector index, a remote reranker, and a filesystem artifact store.
12. Design a typed graph validator that rejects a disclosure effect not dominated by authorization.
13. Create a descriptor catalog schema that supports UI discovery but cannot invoke plugins.
14. Design a generated registry for three typed reranker factories.

Law and testing exercises

15. Write a law suite for a deterministic query rewriter.
16. Write a law suite for a score-fusion plugin, including total ordering under ties and rejection of non-finite scores.
17. Write a law suite for an evaluator that produces recall, MRR, and an authorization constraint.
18. Define certificate invalidation conditions for a provider-backed embedder.
19. Build a dynamic echo handler and adapt it to a typed kernel. Add tests for wrong input/output schema IDs.
20. Construct a broken factory whose bound kernel exposes a different spec. Verify that the binder rejects it.
21. Differential-test a cached component against an uncached reference interpreter.

Architecture exercises

22. Draw the package dependency graph for a system with `core`, `plugin`, `experiment`, `evidence`, `campaign`, and two product domains. Identify cycles and move interfaces to eliminate them.
23. Design lifecycle scopes for plugins: singleton, release-scoped, run-scoped, cell-scoped, and call-scoped. Give one example of each.

24. Define how credentials rotate without changing semantic identity or invalidating retained evidence.
25. Design an out-of-process evaluator protocol that receives hidden labels but cannot read candidate implementation assets.
26. Extend the resource algebra with peak memory and remote disclosure sets. Update **Compose** and **Tensor** semantics.
27. Specify a replay interpreter for remote provider responses and its refinement relation to live execution.

Research exercises

28. Compare the factory design with algebraic effects and handlers. Which responsibilities are semantic operations and which are interpreter handlers?
29. Express a small plugin signature as a free symmetric monoidal category. What equations should realizations preserve?
30. Investigate decorated or structured cospans as a model of open plugin graphs. What additional value would they provide over typed ports in this architecture?

Chapter 4. RAG Optimization as a Compositional Field

Learning objectives

By the end of this chapter, you should be able to:

1. model RAG indexing and querying as one parametrized open process;
2. factor RAG parameters by build-time and query-time ownership;
3. define typed plugin interfaces for chunking, representations, embedding, indexing, retrieval, authorization, fusion, reranking, answer generation, and agent behavior;
4. derive artifact invalidation and reuse from the process graph;
5. construct product-owned retrieval, answer, session, and operational evaluators;
6. run a paired campaign whose core packages do not import RAG types;
7. interpret ordered gate and Pareto results from the companion sandbox;
8. map the reference architecture onto `ragkit`, `ragopt`, GEC, RAG-TTC, and Garden;
9. state the additional refinements required for production releases, distributed execution, and self-optimization;
10. design a capstone optimization campaign with explicit semantics and safety boundaries.

4.1 Why RAG is a demanding test domain

A useful optimization backbone should handle more than a fixed function with numeric parameters. RAG is demanding because its behavior spans at least two phases.

Indexing phase. A corpus is captured, normalized, chunked, represented, embedded, and organized into lexical/vector artifacts. Many interventions require rebuilding expensive derived state.

Query phase. A subject and request are rewritten, routed, searched, filtered, fused, reranked, hydrated, admitted as evidence, and possibly passed to an answer generator or agent.

The phases are coupled. Smaller chunks alter the index and the evidence units seen by answer generation. A new embedding model changes vector search but not lexical terms. A fusion-weight change requires no rebuild. A context policy can change answer quality without changing retrieval metrics. A reranker can improve relevance while adding remote disclosure, stochastic failure, latency, and cost.

Motivation. A RAG optimizer must understand the composition and dependency structure of the entire system. Treating indexing as a preprocessing script and querying as an unrelated service loses the causal meaning of interventions.

4.2 RAG as a parametrized open process

4.2.1 Build and query families

Definition (Release). A release is an immutable, verified boundary object that binds the artifacts and semantic policies required to serve a declared RAG behavior. In the toy system it is an index value; in production it also includes corpus lineage, query policy, reranker, prompts, validators, and structured stores.

Let the build family be

$$B: P_B \otimes \text{CorpusSnapshot} \rightarrow J(\text{Release}),$$

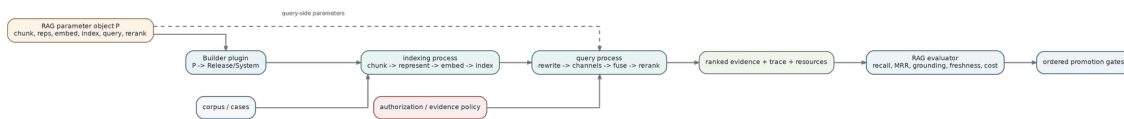
and the query family be

$$Q: P_Q \otimes (\text{Release} \otimes \text{Request}) \rightarrow J(\text{QueryOutcome}).$$

A complete retrieval system is their composition, with the built release wired into query serving:

$$S: (P_B \otimes P_Q) \otimes (\text{CorpusSnapshot} \otimes \text{Request}) \rightarrow J(\text{QueryOutcome}).$$

The release is an internal boundary object. It can be cached, reused, verified, and compared independently.



RAG indexing, query serving, evaluation, and campaign control graft onto the generic backbone.

4.2.2 Open-system interpretation

Definition (Open process). An open process is a component with explicit typed boundary ports through which it exchanges values, capabilities, and observations with an environment. Its internal implementation can change while the boundary semantics remain stable.

RAG is *open* because it interacts with an environment:

- corpus sources and revisions;
- model providers;
- artifact stores;
- authorization policy;
- users and frontend sessions;
- evaluators and campaign control.

The typed ports expose these interactions without making them global dependencies. A build plugin may require an embedding-provider capability. A query plugin may require an authorized release lease. An evaluator may require hidden labels. A proposer may receive only aggregate evidence.

Definition 4.1 (RAG optimization field). A RAG optimization field is an optimization field whose system family factors through typed build, release, query, answer/agent, and presentation processes, and whose parameter/dependency graph records how interventions invalidate artifacts and evaluation levels.

Applied example. A fusion-weight campaign instantiates the same build release with several query parameter values and evaluates retrieval cells. A chunking campaign changes the build family, constructs new releases, then evaluates the same query workload. Both are instances of one field because the build/query boundary and evaluator contracts are explicit.

4.2.3 Parameter factorization

A useful factorization is

$$P = P_{source} \otimes P_{norm} \otimes P_{chunk} \otimes P_{repr} \otimes P_{embed} \otimes P_{index} \otimes P_{query} \otimes P_{rerank} \otimes P_{context} \otimes P_{answer} \otimes P_{agent}.$$

Not every campaign exposes every factor. A candidate is a point in a constrained subspace with locked assets.

Definition 4.2 (Intervention cut). An intervention cut is the subset of parameter factors permitted to change in one campaign. All other factors and evaluator assets are locked.

Applied example. A reranker-only cut permits model revision, pool size, blend, and fallback to vary while chunking, indexes, cases, judge, and gate remain locked. A candidate that also changes the answer judge is rejected as malformed before execution.

A fusion campaign varies P_{query} over a fixed release. A chunking campaign varies P_{chunk} and rebuilds every downstream derivation. A prompt campaign varies P_{answer} while keeping retrieval evidence fixed or deliberately re-executing the full answer path.

4.2.4 Why one candidate should not mean “arbitrary repository diff”

An arbitrary diff can change system behavior, evaluator, gate, data, and reporting simultaneously. Attribution becomes impossible. A candidate should be a canonical semantic patch over one or a small declared set of factors, plus immutable parent and hypothesis.

The companion sandbox uses:

```
type Params struct {
  ChunkWords      int
  OverlapWords    int
  TitlePrefix     bool
  VectorDimensions int
  LexicalWeight   float64
  VectorWeight    float64
  RerankTopN      int
  RerankWeight    float64
  RerankNoise     float64
  TopK            int
}
```

This type intentionally crosses the build/query boundary so the demonstration cannot optimize only a post-hoc ranking formula.

4.3 Domain values and authority

4.3.1 Source documents and chunks

The toy domain begins with:

```
type Document struct {
    ID      string
    Title   string
    Text    string
    Roles []string
}

type Chunk struct {
    ID           string
    DocumentID   string
    Title        string
    Text         string
    StartWord    int
    EndWord      int
    Roles        []string
}
```

A chunk is authoritative evidence because it retains source document identity and exact range in the toy model. In a production system, it should additionally retain source revision, byte or structural span, content digest, and release lineage.

4.3.2 Representations are searchable derivatives

A title-prefixed text, summary, generated question, or contextualized passage can improve retrieval. It is a **representation**, not automatically evidence.

Definition 4.3 (Search representation). A search representation is derived material indexed for retrieval and linked to one or more authoritative evidence objects. It may contribute to ranking, but answer citations resolve to source evidence unless product policy explicitly grants the representation authority.

This distinction prevents a generated summary from becoming an untraceable source of truth.

4.3.3 Subject and authorization

The toy query contains a role:

```
type Query struct {
    Text string
    Role string
}
```

Production subject context should be server-owned and richer: tenant, roles, scopes, purpose, data class, and policy revision. The model or user may supply a query but cannot grant itself authority.

Definition 4.4 (Authorization-first retrieval). A retrieval process is authorization-first when no candidate payload crosses a protected local or remote stage unless it has been admitted by the subject's policy for that purpose and release.

The toy implementation filters role-incompatible chunks before lexical/vector scoring and stochastic reranking. This is deliberately stronger than post-ranking result filtering.

4.4 Indexing plugins

4.4.1 Chunker

A minimal domain interface is:

```
type Chunker interface {
    Spec() core.Spec
    Chunk(Document) ([]Chunk, error)
}
```

The toy `wordWindowChunker` uses size and overlap. Its spec declares `document_local`, which supports incremental invalidation: changing one document need not alter chunks of another.

Production laws should include:

- deterministic output for the same document revision and spec;

- valid, ordered, nonempty source spans;
- source-text reconstruction;
- stable identities;
- no cross-document influence when `document_local` is claimed;
- bounded amplification;
- explicit handling of empty or malformed documents.

Worked example 4.1 (Chunk count). A document has 50 words. A window size of 14 and overlap of 3 yields step $14 - 3 = 11$. Starts are 0, 11, 22, 33, and 44, so five chunks are produced. A size of 26 with no overlap yields starts 0 and 26, so two chunks. The smaller-overlap candidate increases index units and may improve localization.

4.4.2 Representer

```
type Representer interface {
    Spec() core.Spec
    Text(Chunk) string
}
```

The toy representer optionally prefixes the title. A production representer may be stochastic and provider-backed:

```
type Representer interface {
    Factory[Chunk, []Representation]
}
```

Its semantic key includes prompt, model revision, decoding, source chunk, and representation kind. Generated output is retained as material so downstream embedding can be reproduced.

4.4.3 Embedder

```
type Embedder interface {
    Spec() core.Spec
    Embed(string) []float64
}
```

The toy embedder uses deterministic feature hashing. A production interface should return instrumented outcomes and verify dimension, finite values, model revision, normalization, and usage.

```
type EmbedRequest struct {
    RepresentationID string
    Text            string
}

type Embedding struct {
    RepresentationID string
    ModelID         string
    Values          []float32
}
```

4.4.4 Index builder

The toy build composes chunker, representer, and embedder over a sorted corpus. Its index spec includes corpus ID and component spec IDs:

```
spec := core.MustSpec(
    "rag.index",
    "toy_hybrid",
    "v1",
    map[string]any{
        "corpus_id": corpusID,
        "chunker":   chunker.Spec().ID,
        "representer": representer.Spec().ID,
        "embedder":   embedder.Spec().ID,
        "chunk_count": len(chunks),
    },
    []core.Effect{core.EffectPure},
    []string{"lexical", "vector", "authorization_prefilter"},
    ""
)
```

A production index builder should produce an immutable verified artifact or logical release view. Its capability declaration describes exact/approximate search, filter pushdown, upsert/delete, snapshot reads, compaction, and deterministic build class.

4.4.5 Build family and candidate builder

The generic campaign interface is:

```

type Builder[P, X, Y any] interface {
    Spec() core.Spec
    Build(
        context.Context,
        Candidate[P],
    ) (core.Kernel[X, Y], error)
}

```

The toy `ragtoy.Builder` receives a corpus, validates candidate parameters, constructs chunker/representer/embedder, builds the index, and returns a query kernel.

This is one compact implementation of

$$P \rightarrow (X \rightarrow D(J(Y))),$$

which is the curried form of a parametrized process $P \otimes X \rightarrow D(J(Y))$.

4.5 Query plugins

4.5.1 Query rewriting

A rewriter has a port such as:

```

type RewriteInput struct {
    Query    Query
    History  []Message
}

type RewriteOutput struct {
    Queries []string
    Route   string
}

```

It may be deterministic synonyms, stochastic multi-query generation, HyDE, or an intent router. Its trace must record fallback to original query. Hidden labels are not inputs.

4.5.2 Channel searchers

A channel searcher is typed over a release-bound index snapshot:

```

type SearchRequest struct {
    Query      string
    Subject    SubjectContext
    CandidateTop int
}

type Ranking struct {
    Channel string
    Items  []CandidateRef
}

```

Lexical and vector channels can be combined with Fanout and Tensor. Their results retain channel contribution and stable candidate identity.

4.5.3 Authorization

Authorization should be a deterministic policy process with a typed certificate:

```

type AuthorizationInput struct {
    Subject    SubjectContext
    Candidates []CandidateMetadata
    Purpose    string
    ReleaseID  string
}

type AuthorizedCandidates struct {
    Items      []CandidateRef
    Certificate AuthorizationCertificate
}

```

A remote reranker accepts `AuthorizedCandidates`, not raw candidates. The type encodes the ordering constraint discussed in Chapter 3.

4.5.4 Collapse and fusion

Representations may produce several hits for one source chunk or document. Collapse chooses one logical evidence candidate while retaining contributions. Fusion combines channel rankings.

For reciprocal-rank fusion with channel weight w_c and rank constant k :

$$RRF(d) = \sum_c \frac{w_c}{k + rank_c(d)}.$$

A fusion plugin must define:

- missing-channel behavior;
- duplicate/collapse identity;
- weight normalization;
- finite-score validation;
- deterministic total tie order;
- contribution trace;
- whether raw scores or ranks are used.

4.5.5 Reranking

The toy reranker adjusts top candidates using a phrase signal plus paired random noise:

$$score'_i = (1 - \lambda)score_i + \lambda(semantic_i + \epsilon_i).$$

The seed is derived from the query process spec, label reranker, and query text. Authorization already removed forbidden chunks.

A production reranker plugin declares model revision, pool size, text composition, disclosure, timeout, fallback, and random coupling. A failure that falls back to fused order is a degraded outcome, not invisible success.

4.5.6 Evidence admission

A context policy maps ranked candidates to admitted evidence under count, token, diversity, and source-authority constraints. It should retain reasons for inclusion and exclusion. Whole-evidence admission is often easier to validate than arbitrary text truncation.

4.5.7 The toy query kernel

The sandbox `NewSearchKernel` performs:

1. query validation;
2. role authorization prefilter;
3. lexical and hashed-vector scoring;
4. deterministic sorting;
5. optional stochastic reranking;
6. document collapse;
7. top- k emission;
8. deterministic cost model plus paired jitter;
9. structured trace and resource output.

Its trace is:

```
authorize_prefilter
-> score_channels
-> optional_rerank
-> collapse_emit
```

Its capabilities include `authorization_prefilter`, `hybrid`, and `collapse_document`.

4.6 Answer and agent plugins

4.6.1 Retrieve-then-generate answer

A bounded answer process can be written

$$A: P_A \otimes (Query \otimes Evidence) \rightarrow J(Validated Answer).$$

It contains context formatting, provider generation, schema parsing, citation validation, repair/fallback, and resource trace. The evaluator remains separate and can judge relevance, completeness, style, or product utility.

4.6.2 Grounding validator

A generic validator can enforce structural properties:

- every citation resolves to admitted evidence;
- cited evidence belongs to the release and subject;
- answer schema is valid;
- abstention has an allowed reason;
- no unknown evidence labels appear.

These are system contracts and hard constraint observations. They do not replace answer-quality evaluation.

4.6.3 Agent process

An agentic RAG system is not one retrieve-then-generate call. It is a bounded state transition:

$$State_t \xrightarrow{ModelPolicy} Action_t \xrightarrow{ToolInterpreter} State_{t+1}.$$

The process terminates on a final answer, cancellation, failure, or iteration budget. A turn trace contains model decisions, tool call IDs, search outcomes, evidence-session updates, and final validation.

Agent parameters include tool descriptions, route policy, maximum iterations, evidence reuse, retry, and prompt. Evaluating them requires trajectory/session observations, not only final answer text.

4.6.4 Presentation plugins

Garden-like systems project evidence into source cards, product comparisons, choices, or widgets. Presentation can be modeled as

$$P_{ui}: EvidenceSession \otimes AnswerState \rightarrow J(FrontendEvents).$$

Product schemas remain outside the generic core, but event identity, release lineage, and provenance laws can be shared.

4.7 Evaluator plugins for RAG

4.7.1 Retrieval evaluator

The toy evaluator reads hidden relevant document IDs and computes recall, MRR, and precision. It also independently checks authorization and finite scores.

```
type Evaluator[X, Y any] interface {
  Spec() core.Spec
  Evaluate(
    context.Context,
    experiment.EvalRequest[X, Y],
  ) (evidence.Observation, error)
}
```

For relevant set R and returned list H_k :

$$Recall @ k = \frac{|R \cap H_k|}{|R|},$$

$$Precision @ k = \frac{|R \cap H_k|}{|H_k|},$$

$$MRR = \begin{cases} 1/r, & \text{first relevant item at rank } r, \\ 0, & \text{none returned.} \end{cases}$$

The evaluator assigns zeros when a failed outcome has no value, rather than dropping the cell, and retains resource observations.

4.7.2 Answer evaluator

An answer evaluator may combine:

- deterministic citation/contract checks;
- claim-to-evidence entailment;
- required-point coverage;
- contradiction and unsupported-claim detection;
- calibrated model judge dimensions;
- abstention appropriateness;
- human review.

Its judge prompt, model, decoding, and rubric are locked evaluator assets, not candidate parameters unless the campaign explicitly studies evaluation itself.

4.7.3 Session evaluator

A session evaluator operates on a trajectory artifact:

- task completion;
- turn and tool-call count;
- invalid/repeated actions;
- evidence epoch consistency;
- choice validity;
- widget grounding;
- user-visible terminal state;
- latency and cost.

One cell is one conversation scenario at one repeat and arm.

4.7.4 Operational evaluator

An operational evaluator analyzes build and serving behavior:

- build duration and cache reuse;
- index size and memory;
- query p95/p99 latency;
- failure/fallback rate;
- freshness lag;
- throughput and queue behavior;
- crash/resume equivalence;
- remote disclosure.

A candidate that changes an index backend or deadline needs this fidelity even when retrieval quality is unchanged.

4.8 Dependency closure and artifact reuse

4.8.1 The dependency graph

A production graph can contain nodes:

```
source snapshot
-> normalized documents
-> chunks
-> representations
-> embeddings
-> lexical index
-> vector index
-> release
-> channel rankings
-> authorized candidates
-> fused ranking
-> reranked ranking
-> evidence context
-> answer/trajectory
-> frontend projection
-> evaluation observations
```

Each node has semantic spec and content identity. A candidate patch marks directly changed nodes; the planner computes downstream closure.

Definition 4.5 (Invalidation closure). Given a dependency graph and a set of changed semantic nodes U , the invalidation closure is the least set containing U and every node whose denotation depends on a node in the set.

Applied example. Changing a summary-generation prompt invalidates generated summaries, their embeddings, affected index entries, query outcomes that use those entries, and downstream answers. It does not invalidate source capture, normalization, or raw chunks.

4.8.2 Reuse examples

- Fusion-weight change reuses source, chunks, representations, embeddings, indexes, and optionally retained channel rankings.
- Rerank-pool change reuses channel rankings but reruns authorization, hydration, reranking, context, and downstream evaluation.
- Title-prefix representation change reuses source and chunks, but regenerates representation text, embeddings, indexes, and downstream outcomes.
- Chunk-size change invalidates chunks and everything downstream.
- Answer-prompt change reuses evidence only when the campaign claims fixed-evidence answer evaluation; full end-to-end behavior requires query re-execution if stochastic coupling or context policy can interact.

4.8.3 Semantic cache keys

A cache key is derived from operation spec and typed input identity:

$$K = H(\text{operationSpecID}, \text{inputArtifactIDs}, \text{schema}).$$

Provider calls with stochastic resampling need a sample coordinate or retained-response identity. Caches should not be keyed by mutable path or plugin-owned ad hoc strings.

4.8.4 Artifact authority

Definition (Artifact). An artifact is immutable retained material with a verifiable identity, schema, and provenance, such as an index, provider response, query trace, native evaluation record, or report. A mutable path is a locator, not artifact identity.

Applied example. `s3://bucket/current/index.json` is a mutable locator. The artifact is the verified manifest and child objects identified by content digests. A cell should retain the artifact reference, while deployment configuration may retain the locator used to resolve it.

Every shared metric should link to a native artifact containing enough detail for diagnosis. For RAG, that may include:

- release and component specs;
- query and subject projection;
- channel rankings and contributions;
- authorization decisions;
- reranker request/response references;
- final evidence and answer;
- trace, usage, failure, and evaluator diagnostics.

The shared `Observation` is a projection, not the full truth.

4.9 The companion sandbox campaign

4.9.1 Corpus and cases

The demo corpus contains eight short tree-care documents. Seven are public/admin. One restricted document describes an internal inventory override and is admin-only.

The ten cases include ordinary care queries, a multi-relevant query, and two identical security queries under different roles:

- public inventory-override query expects no restricted result;
- admin inventory-override query expects the restricted document.

This pair tests that authorization is part of the system behavior, not a post-hoc evaluator filter.

4.9.2 Baseline

The baseline uses:

```
chunk words:      26
chunk overlap:    0
title prefix:     false
vector dimensions: 24
lexical weight:   0.72
vector weight:    0.28
rerank top N:     0
top K:            3
```

4.9.3 Candidates

The five challenger hypotheses are:

1. **smaller-overlap**: smaller overlapping chunks improve localization;
2. **title-hybrid**: title-aware representations improve intent matching;
3. **bounded-rerank**: a small stochastic reranker improves first relevant rank;
4. **expensive-wide**: more vector dimensions and reranking improve quality at higher cost;
5. **lexical-heavy**: lexical dominance helps exact terminology.

Each candidate is a canonical typed parameter value with parent and hypothesis.

4.9.4 Exact law certificate

Before the campaign, the demo certifies the exact finite-process laws from Chapter 1:

Law	Result
left identity	pass
right identity	pass
associativity	pass
discard naturality	pass
deterministic copy naturality	pass
copy commutativity	pass

It also certifies deterministic chunking for the selected chunker fixture and registers the spec/certificate in the descriptor catalog.

4.9.5 Experiment configuration

The campaign uses:

- 10 cases;
- 5 repeats;
- exact baseline/candidate pairs;
- root seed 20260809;
- shared semantic random namespaces;
- one JSONL ledger per incumbent/challenger trial;
- retrieval evaluator;
- ordered gate policy;
- best-MRR selector among eligible candidates.

Each challenger therefore yields 50 baseline cells and 50 candidate cells, paired by case and repeat.

4.9.6 Gate program

The gate order is:

1. every cell satisfies authorization;
2. every score is finite;
3. recall is noninferior with zero margin;
4. MRR is noninferior with margin 0.03;
5. mean latency is at most 1.3 ms in the toy resource model;
6. mean index units are at most 1200;
7. MRR improvement is at least 0.005 as a soft requirement.

The numbers are pedagogical, not production recommendations. The structure is the lesson.

4.10 Running the sandbox

From the module directory:

```
go test ./...
go test -race ./...
go run ./cmd/probopt-demo \
  -out out \
  -seed 20260809 \
  -repeats 5
```

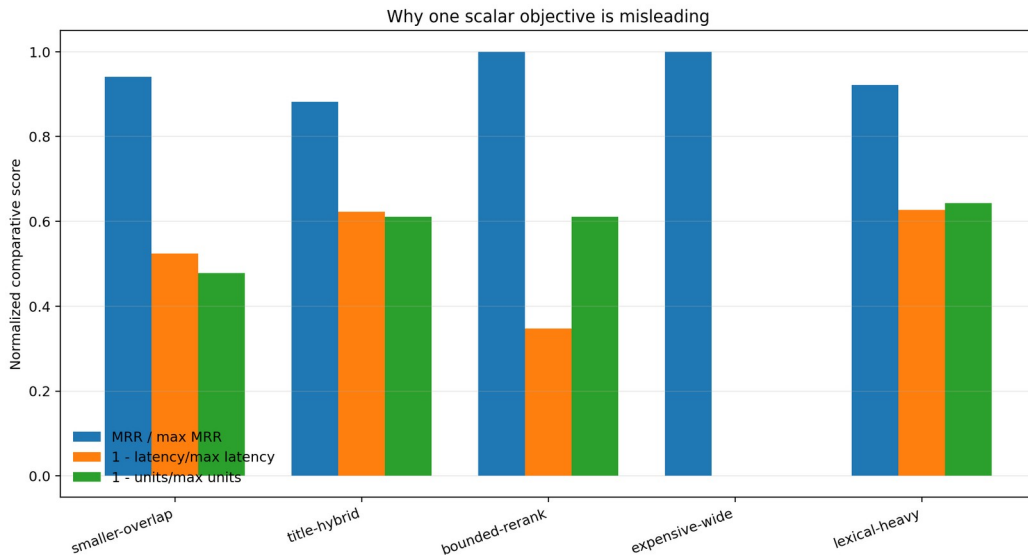
The run writes:

```
out/report.md
out/report.json
out/pareto.csv
out/runs/*/cells.jsonl
```

A second invocation validates and reuses existing cell coordinates.

The generated report contains this candidate summary:

Candidate	Verdict	Recall	MRR	Precision	Latency ms	Index units	Pareto
bounded-rerank	eligible	0.900	0.850	0.333	1.013	688	yes
expensive-wide	rejected	0.900	0.850	0.333	1.551	1768	no
lexical-heavy	eligible	0.900	0.783	0.333	0.579	631	yes
smaller-overlap	eligible	0.900	0.800	0.333	0.739	922	yes
title-hybrid	undecided	0.900	0.750	0.333	0.586	688	no



Quality, latency, and index-size trade-offs in the sandbox campaign.

4.10.1 Interpreting bounded-rerank

bounded-rerank improves mean MRR by 0.10, preserves recall, passes authorization and finite-score constraints, and remains within latency/index budgets. It is eligible and selected as the new incumbent by the configured selector.

This does not prove universal superiority. It proves eligibility under the demo workload, resource model, evaluator, coupling, and gate.

4.10.2 Interpreting expensive-wide

expensive-wide obtains the same recall and MRR as bounded-rerank but exceeds latency budget; gate evaluation stops at the failed hard rule. It is rejected despite quality.

This demonstrates why resource constraints are not a postscript. A candidate can be quality-equivalent and operationally inadmissible.

4.10.3 Interpreting title-hybrid

title-hybrid passes hard constraints and budgets but shows no MRR improvement. The soft improvement rule fails, producing Undecided. This is semantically different from rejection: the candidate is not unsafe or clearly inferior; it simply lacks the required reason to promote.

4.10.4 Pareto front

bounded-rerank, lexical-heavy, and smaller-overlap are on the demo Pareto front. Each represents a different trade-off among MRR, latency, and index size. title-hybrid is dominated or lacks target improvement. expensive-wide is removed before Pareto analysis because it fails a hard budget.

The selector chooses maximum MRR among eligible candidates. Another product policy might choose lexical-heavy for lower latency or retain several cohort-specific releases.

4.11 Walking through the code path

4.11.1 Candidate construction

```
initial := campaign.MustCandidate(
  "baseline",
  "",
  "current deterministic hybrid baseline",
  ragtoy.BaselineParams(),
)
```

Each challenger is created with parent `initial.ID`, hypothesis, and typed Params. Candidate identity derives from canonical content.

4.11.2 Builder

```
builder := ragtoy.NewBuilder(ragtoy.DemoCorpus())
```

For each candidate, Build creates component specs and index artifacts, then returns a query kernel. The generic campaign sees only Builder[Params, Query, SearchResult].

4.11.3 Evaluator

```
evaluator := ragtoy.NewEvaluator(ragtoy.DemoCorpus())
```

The evaluator receives hidden expected IDs through experiment.Case.Expected; the query kernel receives only Case.Input.

4.11.4 Runner and ledger

The campaign creates a file ledger per trial. The experiment runner derives exact seeds, executes baseline and candidate, evaluates both, commits cells, aggregates summaries, and produces paired deltas.

4.11.5 Gate and selector

The generic evidence package applies the ordered RAG-independent rules. The selector chooses the best eligible MRR. Neither package imports ragtoy.

4.11.6 History

The final history records initial candidate, selected incumbent, all trials, summaries, and verdict evidence. Historical cells are immutable. A new campaign round can propose candidates relative to the new incumbent without rewriting the past.

4.12 From the sandbox to ragkit and ragopt

4.12.1 Preserve existing strengths

A pragmatic migration should not discard existing experiment custody. ragopt already owns immutable candidate snapshots, exact case/repeat/arm coordinates, run custody, comparison, gates, and promotion reports. The compositional backbone strengthens what an arm means.

A near-term integration is:

1. build a typed ragkit process and component specs;
2. execute one product case under stable seed namespaces;
3. retain the complete trace and native product artifact;
4. project metrics, failures, and usage into the existing ragopt outcome boundary;
5. let the existing runner own durable pairing and reports.

The companion compat/ragoptv1 package demonstrates a JSON-compatible projection without importing the original repository.

4.12.2 Suggested package boundaries

A target layout can be:

ragopt/	
sem/	process specs, outcomes, traces, stable randomness
evidence/	metric schemas, paired analysis, gates, Pareto
plugin/	factories, policies, certificates, catalogs
control/	proposers and selectors
ragspace/	RAG parameters and dependency closure
ragkit/	
corpus/	documents, revisions, chunks, representations
build/	typed build processes and release artifacts
query/	search plans and interpreters
answer/	context and grounded answer processes
agent/	bounded tool-loop integration
eval/	RAG-native evaluator interfaces and law suites

The exact repository split can differ. The dependency rule matters: generic ragopt core does not own RAG domain types; ragspace and product arms graft them on.

4.12.3 Component extraction sequence

A conservative sequence is:

1. introduce canonical component specs and semantic IDs;
2. add stable random namespaces and structured trace references;
3. type metric directions, constraints, and finite-value rules;
4. wrap existing `ragkit` components in factories without changing runtime behavior;
5. add law suites for ordering, lineage, authorization, cache soundness, and backend refinement;
6. split product evaluators from opaque arm callbacks;
7. introduce typed RAG parameter spaces and dependency closure;
8. add proposer/selector plugins only after evidence semantics are stable.

Mathematical vocabulary without stable identity and custody would not improve the system. The extraction order starts with executable contracts.



A staged migration introduces semantic identity and typed grafts before adaptive control.

4.13 Mapping to the applied systems

4.13.1 GEC

GEC's knowledge service can become a product facade over typed query plans. Product-owned pieces remain:

- access scopes and source roles;
- curated lexical synonyms;
- remote reranker text composition and fail-open policy;
- administrative tool schema;
- evidence-label presentation;
- answer judge and protected strata.

Shared semantics should make authorization-before-disclosure a graph law, bind synonyms/reranker into release identity, emit structured traces, and expose current sweeps as typed candidates under `ragopt` custody.

4.13.2 RAG-TTC

RAG-TTC should delete copied common RAG implementation after differential fixtures and use `ragkit` as the sole shared substrate. Product-specific connected retrieval, product catalog, agent tools, providers, diagnostics, and native evaluators remain in RAG-TTC.

Its ANN bakeoff becomes an evaluator of an approximation-changing index plugin against an exact interpreter. Its tool-evaluation adapter is already close to the desired graft: product execution and native judge remain local while `ragopt` owns paired cells and gates.

4.13.3 Garden

Garden's system output includes intent, choices, structured facts, source cards, product comparisons, and widgets. These remain product-owned presentation semantics. Shared process/evidence layers provide release-pinned evidence references, effects, traces, exact scenario/repeat cells, and campaign custody.

A Garden evaluator maps a native session artifact to task completion, field-level provenance, conflict suppression, interaction count, latency, and cost.

4.14 Production refinements beyond the sandbox

The sandbox is a semantic reference, not a distributed production engine. A production implementation adds the following without redefining the core concepts.

4.14.1 Behavior-complete releases

A release should bind corpus snapshot, chunks, representations, embeddings, indexes, query policy, reranker, prompts, validators, structured stores, and presentation policy. A query or turn obtains one release lease.

The build process becomes

$$B: P_B \otimes CorpusSnapshot \rightarrow J(VerifiedRelease),$$

and activation is a separate compare-and-swap effect outside the optimization campaign.

4.14.2 Durable distributed custody

JSONL is sufficient for the sandbox. Production needs transactional or append-only stores with:

- run and cell fencing;
- idempotent commit;
- artifact verification;
- lease/retry semantics;
- cancellation and terminal run state;

- exact resume after process loss;
- access control and retention.

The cell semantics stay the same.

4.14.3 Remote provider refinement

Live provider calls should have replay artifacts or response digests, immutable model revision where available, timeout/fallback policy, disclosure certificate, and resource trace. A replay interpreter can reproduce experimental evaluation without another provider call.

4.14.4 Continuous and large probability spaces

The sampled kernel API represents one draw without implementing measure theory. Statistical libraries can analyze retained cells. If exact measure-theoretic composition becomes necessary, a different interpreter can realize the same typed signatures using appropriate probability objects.

4.14.5 Real concurrency and resource algebra

Production tensor can use goroutines, async RPC, batching, or distributed workers. Semantic trace remains a canonical parallel tree. Resource dimensions declare additive, critical-path, peak, or set-union composition. Performance traces can retain operational timing separately.

4.14.6 Security and information flow

Effect tags are strengthened by typed authorization certificates, provider data-policy checks, sandboxed dynamic evaluators, and visibility-specific artifact APIs. System, evaluator, proposer, and decision authority receive different capabilities.

4.14.7 Deployment remains outside campaign selection

An eligible candidate is not automatically deployed. Promotion produces evidence and a release reference. An activation authority applies canary, rollback, and operational policy. This boundary prevents a proposer or selector from mutating production directly.

4.15 Self-optimization without self-authorization

An LLM can be a useful proposer, code generator, failure analyst, or evaluator assistant. The semantic architecture constrains its authority.

A self-optimization loop may:

1. read proposer-visible campaign history;
2. generate a typed candidate patch and hypothesis;
3. pass schema, capability, and locked-asset validation;
4. build through typed component factories;
5. execute paired product-owned evaluators;
6. pass ordered hard gates and holdout policy;
7. produce a promotion report for independent activation authority.

It may not:

- rewrite its hidden evaluator or hard gate unnoticed;
- read protected holdout cases unless policy permits;
- omit failed cells;
- grant itself capabilities or access scopes;
- choose a new provider policy outside candidate schema;
- deploy itself because it selected itself.

Definition 4.6 (Self-optimization boundary). A self-optimizing controller may adapt candidate proposals from authorized evidence, but it cannot redefine system semantics, evaluator identity, decision policy, or deployment authority outside an explicit higher-level intervention.

Applied example. An LLM may propose a new query-rewrite prompt and explain its hypothesis. The candidate schema locks the judge and security gate; paired evaluation runs independently; an activation service requires a signed promotion report. The LLM can influence the proposal but cannot declare itself correct or deploy the result.

This boundary makes self-optimization an instance of the same control semantics rather than a privileged exception.

4.16 A capstone design method

Use the following sequence to design a real optimization campaign.

Step 1: draw the system graph

List typed ports from source/corpus through user-visible outcome. Mark sequential, parallel, and data-dependent control. Separate build, query, answer/agent, and presentation.

Step 2: factor parameters

Assign each behaviorally material configuration field to a parameter object and component spec. Identify locked assets and candidate intervention cut.

Step 3: classify effects and capabilities

Mark randomness, local/remote I/O, disclosure, state, filter pushdown, exact/approximate behavior, streaming, and idempotency.

Step 4: define outcomes and trace

State product failure classes, degradation paths, cancellation, resources, semantic events, artifact lineage, and visibility projections.

Step 5: define cases and evaluator boundary

Split visible input from hidden labels/rubrics. Define native artifact and observation schema. Identify evaluation fidelity: retrieval, answer, session, operational, or frontend.

Step 6: choose coupling and coordinates

Define case/repeat/arm identity, root seed, semantic shared namespaces, provider replay, execution order, and missingness policy.

Step 7: define gates before running

Order security, integrity, reliability, quality, cost, and target-improvement rules. State quantifiers, margins, statistics, protected strata, and holdout policy.

Step 8: define candidate search and visibility

Choose proposer inputs, candidate schema, budget, stopping, and selector policy. Keep proposal separate from evidence and deployment.

Step 9: certify plugins and interpreters

Run exact laws, property tests, differential fixtures, authorization spies, backend refinement, and interruption/resume tests.

Step 10: retain and review evidence

Store raw cells, native artifacts, summaries, gate evidence, Pareto front, selected candidate, and activation plan. Make the decision reproducible without rerunning live providers where possible.

4.17 Chapter summary

RAG becomes compositional when indexing and querying are modeled as one parametrized open process with a release boundary. Build and query parameter factors determine artifact invalidation and reuse. Domain plugin interfaces preserve the meaning of chunks, representations, embeddings, indexes, authorization, rankings, evidence, answers, agents, and presentation without moving those concepts into the generic optimization core.

The companion sandbox demonstrates the complete graft. A generic campaign builder realizes typed RAG candidates. A product evaluator sees hidden labels. Exact paired cells use stable randomness and durable custody. Ordered gates reject unsafe or over-budget candidates, distinguish undecided from rejected, compute a Pareto front, and select an eligible incumbent. Generic packages do not import RAG.

Production systems refine the same semantics with behavior-complete releases, distributed ledgers, provider replay, typed authorization, real resource algebras, and independent deployment authority. Self-optimization is permitted at the control layer without allowing the controller to authorize or grade itself.

4.18 Exercises and capstone projects

Conceptual exercises

- Factor the parameter object of a RAG system you know into source, build, query, answer/agent, and presentation factors.
- For each factor, identify the earliest affected process node and its invalidation closure.
- Explain why a generated summary representation is not automatically evidence.
- Give an example where authorization after reranking returns correct visible results but violates process semantics.
- Distinguish a release artifact from a deployment instance and from a campaign candidate.
- Give one retrieval metric improvement that can harm answer quality and one answer improvement that can harm frontend utility.
- Explain why an agent tool description belongs in semantic identity.

Mathematical exercises

- Write the full composite type of build B , query Q , and evaluator E for a retrieval-only campaign.
- Extend it to an answer evaluator and identify which values must be copied or retained.
- Define a parameter reparametrization that maps presets `cheap`, `balanced`, and `quality` into detailed RAG parameters. State identity requirements.
- For RRF with two channels, calculate scores for three documents at given ranks and weights. Analyze how changing rank constant affects shallow versus deep ranks.
- Define a refinement relation for an ANN index relative to exact search, including recall and latency.
- Model an agent turn as a finite-state Markov process with a maximum of three tool calls. Define terminal states.
- Define a Pareto relation over MRR, p95 latency, provider cost, and index size with practical tolerances.

Implementation exercises

- Run the companion sandbox with a different root seed. Which aggregate values change, and which identities remain stable?
- Add a candidate that increases `TopK`. Predict its precision/recall and cost effects before running.
- Add a hard `contract` gate to the demo. Explain how the public restricted query should be represented.
- Add a query-rewrite plugin with its own random namespace and trace event.
- Refactor the toy search into explicit lexical and vector kernels composed with `Fanout` and `Tensor`.
- Add a remote-reranker spy and certificate input. Prove unauthorized text is never disclosed.
- Add a cached embedding interpreter and differential-test it against the uncached embedder.
- Extend the cell artifact with a native RAG trace reference rather than embedding the full output.
- Add bootstrap or permutation analysis to the paired MRR differences without changing the gate interface.
- Add a `CostPerQuery` metric and recompute the Pareto front.
- Add an answer-generation layer and a deterministic citation validator.

Architecture projects

- ragkit graft.** Select three existing `ragkit` components. Define `Spec`, typed factory, effects, capabilities, random namespace, and law suite for each.
- ragopt integration.** Wrap a typed process/evaluator pair behind the current `ragopt` arm boundary while retaining full native trace and projection.
- Incremental build field.** Extend the build family with corpus revisions, delta indexes, and compaction. Define parameter factors, artifacts, and equivalence to full rebuild.
- Agent campaign.** Design a multi-turn Garden-like campaign with choice messages, structured facts, evidence widgets, and release epochs.
- ANN certification.** Implement an exact vector oracle and approximate candidate, then define workload-relative recall, determinism, update, and operational gates.
- Dynamic evaluator.** Implement an out-of-process evaluator protocol with hidden-label isolation, schema versions, capability grant, and law certificate.
- Release manager.** Design compare-and-swap activation and release leases. Explain how campaign eligibility connects to but does not perform deployment.

Capstone: complete optimization field

Design and implement a complete field for one real optimization problem. Your submission should include:

- a typed process graph and parameter factorization;
- denotational signatures for system, evaluator, decision, and control layers;
- an operational state machine for cells and resume;
- canonical specs and semantic identity rules;
- at least three plugin interfaces and law suites;
- explicit effects, capabilities, and visibility classes;
- paired coupling and random namespace design;
- native artifact and observation schemas;
- ordered gate policy and Pareto metrics;
- a reference interpreter and a more realistic refinement;
- failure, cancellation, and missingness tests;
- a reproducible report with raw cell references.

Selected exercise hints

The purpose of these hints is to help a reader restart a stalled derivation without turning the exercises into mechanical substitutions. Complete solutions should state assumptions and preserve the semantic distinctions used in the chapters.

Chapter 1 hints

Exercise 2, kernel composition. Treat the matrices as row-stochastic. The (x, z) entry of the composite is the dot product of row x from K with column z from L .

Exercise 3, copy versus resample. Sample-then-copy has support only on (H, H) and (T, T) . Sample-twice has probabilities p^2 , $p(1-p)$, $(1-p)p$, and $(1-p)^2$. Equality of the joint distributions requires the mixed probabilities to vanish.

Exercise 10, deterministic copy naturality. Evaluate both sides on an arbitrary x . The left side returns $(f(x), f(x))$. The right side first returns (x, x) and then applies f to both components.

Exercise 13, resource algebra. One possible product is

$$R = R_{\geq 0}^{cost} \times R_{\geq 0}^{elapsed} \times R_{\geq 0}^{peak} \times P(DataClass).$$

Use addition for cost, addition sequentially and maximum in parallel for elapsed time, maximum for peak memory, and set union for disclosure.

Exercise 20, authorization spy. Put a unique marker in every unauthorized chunk. The mock reranker should fail immediately if any marker appears in its request. Tests over returned hits alone are insufficient.

Chapter 2 hints

Exercise 8, independent Bernoulli coupling. The four joint probabilities are products of the corresponding marginal probabilities. Sum across rows and columns to recover each marginal.

Exercise 9, maximal positive coupling. Couple shared successes with probability $\min(p_b, p_c)$ and shared failures with probability $\min(1 - p_b, 1 - p_c)$. The remaining mass appears in only one off-diagonal cell.

Exercise 11, summary monoid. Associativity follows componentwise. Minimum and maximum are associative and commutative over nonempty components; use $+\infty$ and $-\infty$ identities or an explicit empty-state flag.

Exercise 14, gate noncommutativity. A hard security rule before an expensive quality analysis can stop early. Reversing them may compute quality for an inadmissible candidate and expose misleading diagnostics. Even when the final verdict matches, operational cost and report semantics differ.

Exercise 17, JSONL ledger. Write each cell as one complete line, synchronize the file, and rebuild an in-memory key index by scanning on open. Reject a line whose run/candidate/spec identity disagrees with the current run.

Exercise 24, local versus remote reranker. The remote candidate changes disclosure, provider failure, latency, and cost in addition to relevance. The campaign therefore needs hard data-policy gates and fault-injection cases, not only ranked-list metrics.

Chapter 3 hints

Exercise 1, core minimality. CandidateID, Trace, ParetoFront, and CellKey satisfy cross-domain custody or decision requirements. Prompt, AccessScope, VectorDimension, and Widget usually remain domain-specific values that can appear inside specs or adapters.

Exercise 9, typed environment. Replace string lookup with small interfaces such as `RerankerClients.Resolve(ProviderID)`, `ArtifactStores.Open(StoreID)`, and `Secrets.Token(CredentialRef)`. Pass only the capabilities needed by the factory.

Exercise 12, disclosure domination. Traverse every path from graph input to a node with `EffectDisclosure`. Require a preceding node that produces an authorization certificate consumed by the disclosure node. A flat effect set cannot express this path property.

Exercise 16, fusion law suite. Include input-order invariance, deterministic tie order, finite scores, contribution conservation, duplicate collapse, and sensitivity to behaviorally material weights.

Exercise 27, replay interpreter. A live call produces and stores a response artifact keyed by semantic request identity and sample coordinate. Replay verifies the artifact and returns the same value/trace projection without a network effect. State which live failure behaviors replay can and cannot reproduce.

Chapter 4 hints

Exercise 8, composite type. Start with

$$B:P_B \otimes \text{Corpus} \rightarrow J(\text{Release}),$$

then thread a successful release into

$$Q:P_Q \otimes (\text{Release} \otimes \text{Request}) \rightarrow J(\text{Hits}).$$

The evaluator also needs the hidden case, and the operational composition must specify what happens when build fails.

Exercise 11, RRF. Compute each channel contribution separately. Increasing the rank constant reduces the relative advantage of rank 1 over deeper ranks and makes channel depth matter differently.

Exercise 12, ANN refinement. A useful relation includes sound authorization/filtering, no deleted items, recall lower bound against exact top- k , finite scores, and a latency/cost improvement. Equality of exact ranks is not expected.

Exercise 19, explicit channels. Define `Lexical: Query -> Ranking` and `Vector: Query -> Ranking`. Use `Fanout(Lexical, Vector)` and then a deterministic fusion kernel. Preserve one query/subject copy if authorization requires it.

Exercise 32, release manager. Linearize activation through compare-and-swap on the routing head. A lease captures the release active at acquisition. A superseded release drains until its lease count reaches zero. Promotion evidence is an input to activation policy, not the activation itself.

Glossary

Adaptive campaign. A campaign in which the proposer or allocation policy chooses future candidates from a bounded view of completed evidence.

Admissible candidate. A candidate that passes every hard gate. Admissibility does not necessarily imply that target improvement has been demonstrated.

Answer interpreter. A process that performs bounded retrieval, context construction, generation, and validation to produce a final answer outcome.

Arm. One role in an experiment, commonly baseline or candidate, bound to a specific candidate and system kernel.

Artifact. Immutable retained material such as an index, provider response, native evaluation record, trace, or report, identified and verified independently of a mutable path.

Authorization-first retrieval. A process ordering in which candidate payload is admitted by subject policy before it reaches any protected local or remote stage.

Binding. Resolution of a pure plugin specification against runtime resources to produce an executable typed kernel.

Capability. A declared property provided or required by a plugin, such as filter pushdown, exact scores, streaming, or authorization certificates.

Campaign history. Immutable state containing the incumbent, candidates, completed trials, verdicts, budgets, and proposer-visible projections.

Candidate. A content-identified parameter value or semantic patch with parent, hypothesis, and locked assets.

Canonical encoding. A deterministic serialization whose treatment of maps, numbers, omitted fields, order, and schema version is precisely specified for identity computation.

Case. A workload item containing system-visible input and evaluator-only expected material, plus identity and tags.

Cell. One durable arm execution and evaluation at an exact run/case/repeat/arm coordinate.

Certificate. Retained evidence that a specification and implementation passed a named law suite under identified fixtures and environment.

Comonoid copy. The deterministic map $\Delta_X : X \rightarrow X \otimes X$ that shares one value into two branches. It is not independent resampling of a stochastic process.

Constraint. An admissibility proposition evaluated under an explicit quantifier. A hard constraint cannot be compensated by another metric.

Control semantics. The meaning of proposal, allocation, stopping, selection, and history update in an adaptive campaign.

Coupling. A joint stochastic process over baseline and candidate outcomes whose marginals equal the individual system processes.

Degraded outcome. A value-producing outcome that followed a declared fallback or quality-reducing path and therefore differs from intended-path success.

Denotational semantics. A mathematical mapping from a specification to the process or distribution of outcomes it denotes, abstracting from execution steps.

Descriptor catalog. A registry of pure specifications, schemas, effects, capabilities, documentation, and certificates. It is not an executable service locator.

Deterministic morphism. A process induced by a function and therefore represented by a Dirac distribution at each input. Deterministic morphisms preserve copying.

Direction-normalized delta. A paired metric difference multiplied by $+1$ for maximize metrics and -1 for minimize metrics so positive always favors the candidate.

Discard. The map $!_X : X \rightarrow I$ that ignores a value. For a normalized stochastic process, discarding output equals discarding input.

Dynamic plugin edge. An explicit out-of-process or schema-erased boundary with versioned codecs, capabilities, effects, trace, and failure contracts, immediately adapted back to typed ports.

Effect. A conservative declaration of actions such as randomness, local I/O, remote I/O, state, or data disclosure.

Eligible. The verdict assigned when hard rules pass and the required promotion evidence is satisfied.

Evaluator. A product-owned process that maps a hidden case and instrumented system outcome to an observation.

Evidence semantics. The meaning of workloads, visibility, couplings, evaluators, cells, observations, aggregation, and durable custody.

Exact interpreter. A realization, such as finite rational stochastic matrices, in which selected semantic laws can be decided exactly.

Extensional outcome. The externally projected value or distribution, excluding some internal facts about how it was produced.

Factory. A typed plugin interface exposing a pure `Spec()` and a `Bind(Environment)` operation that returns an executable kernel.

Failure outcome. An attributable product-level terminal state retained inside the experiment rather than thrown away as a runner error.

Fanout. Copying one deterministic input value and sending it to two processes. In the sandbox it is copy followed by tensor.

Global element. A morphism $I \rightarrow P$, interpreted here as one selected parameter value or candidate.

Hard gate. A decision rule whose failure yields rejection regardless of later metrics.

Hidden label. Expected or rubric material retained in the case for evaluation but excluded from the system-visible projection.

Impact or invalidation closure. The least downstream set of process/artifact nodes whose denotation depends on a changed parameter or input.

Incumbent. The currently selected baseline candidate against which challengers are compared in a campaign round.

Instrumented outcome. A result containing value/failure/cancellation, structured trace, resources, and warnings or fallback information.

Intensional trace. Structured information about component identity, lineage, branches, retries, disclosure, fallback, and other protected facts not necessarily visible in the final value.

Interpreter error. A failure of execution custody or the plugin contract itself, such as corrupt storage or spec mismatch. It is distinct from product failure.

Law certificate. See *certificate*.

Markov category. A symmetric monoidal category with copy and discard structure on each object, used to reason about deterministic data flow and stochastic processes.

Markov kernel. A process assigning an output probability distribution to each input.

Metric specification. A stable definition of a numeric observation including name, direction, unit/domain, finite-value rule, and optional tolerance.

Missing cell. An expected experimental coordinate for which no valid durable cell exists. It is not equivalent to a completed failure.

Monoidal tensor. Parallel composition of processes and their input/output objects. It describes wiring, not necessarily physical threads.

Native artifact. The product-specific detailed record that remains authoritative for diagnosis; shared observations are projections of it.

Noninferiority. A claim that candidate performance is not worse than baseline by more than a declared margin on a protected metric.

Observation. The evaluator's typed result for one cell, including metrics, constraints, status, tags, and artifact references.

Operational semantics. Rules describing how runtime configurations step or evaluate through binding, execution, short circuit, cancellation, evaluation, and commit.

Optimization field. The combined system, evidence, decision, and control structures needed to define and execute optimization without binding them to one proposer.

Parameter object. The typed space P whose values instantiate a family $P \otimes X \rightarrow Y$.

Pareto dominance. A relation in which one admissible candidate is no worse on every selected metric and strictly better on at least one, within declared tolerances.

Pareto front. The admissible candidates not dominated by another admissible candidate.

Paired experiment. An experiment that joins baseline and candidate cells at the same case/repeat coordinate and analyzes paired differences.

Parametrized process. A process $P \otimes X \rightarrow Y$ representing a family of systems indexed by parameter values.

Plugin law. A semantic obligation such as determinism, finite scoring, authorization, lineage preservation, or refinement to a reference interpreter.

Product failure. A declared failure mode of the system under test, represented in its outcome and retained in the experiment.

Proposer. A control plugin that emits candidates or a stop decision from an authorized view of campaign history.

Random namespace. A stable semantic label used with root seed and coordinates to derive a component-local pseudorandom stream.

Reference interpreter. A deliberately simple realization used as an oracle for laws or differential testing.

Refinement. A declared relation under which one interpreter or component is an acceptable implementation of another after a specified projection.

Release. In a RAG system, an immutable behavior-bearing object connecting built corpus/index artifacts with query, answer, and policy configuration.

Repeat. A planned stochastic replicate with stable identity and seed derivation, not an unbounded retry.

Reparametrization. A map $P' \rightarrow P$ translating a public or constrained parameter space into a detailed internal one.

Resource algebra. The operations used to combine cost, latency, memory, disclosure, and other resource dimensions under sequential and parallel composition.

Reranker. A process that reorders or rescales a bounded candidate pool, often with additional model, disclosure, latency, and failure semantics.

Selector. A decision/control component that chooses an eligible successor from completed trials according to product policy.

Semantic identity. A content-derived identifier over every declared field capable of altering a component's denotation or protected trace.

Semantic trace. See *intensional trace*.

Service locator. A registry that returns executable instances by untyped name or key. This book discourages it inside the semantic core.

Soft gate. A rule whose failure does not make the candidate inadmissible but prevents or weakens promotion, producing Undecided.

Specification. The pure canonical description of a component's claimed semantics, effects, capabilities, schemas, and random namespace.

Stable seed derivation. Domain-separated derivation of component-local pseudorandom streams from a root seed and semantic labels rather than mutable generator order.

Statistical stratum. A protected subset of cases or cells, such as role, intent, language, or difficulty, analyzed separately from pooled aggregates.

System semantics. The meaning of the deterministic and stochastic processes being varied and their composition.

Tensor. See *monoidal tensor*.

Three-valued verdict. The decision set Eligible, Undecided, and Rejected.

Trace projection. A visibility-controlled mapping from a rich trace to the fields permitted for a given consumer.

Typed port. A compile-time input/output boundary of a process or plugin, preserving the objects in the semantic graph.

Undecided. The verdict used when hard constraints pass but evidence for required improvement is insufficient or a soft rule fails.

Visibility projection. The deterministic map extracting system-permitted input from a richer case or history object.

Bibliography

Probability, process categories, and stochastic semantics

Fritz, Tobias. 2020. “A Synthetic Approach to Markov Kernels, Conditional Independence and Theorems on Sufficient Statistics.” *Advances in Mathematics* 370: 107239.

Fritz, Tobias, Tomas Gonda, Paolo Perrone, and Eigil Fjeldgren Rischel. 2023. “Representable Markov Categories and Comparison of Statistical Experiments in Categorical Probability.” *Theoretical Computer Science* 961: 113896.

Giry, Michele. 1982. “A Categorical Approach to Probability Theory.” In *Categorical Aspects of Topology and Analysis*, Lecture Notes in Mathematics 915. Springer.

Di Lavore, Elena, and Mario Roman. 2023. “Evidential Decision Theory via Partial Markov Categories.” In *Proceedings of the 38th Annual ACM/IEEE Symposium on Logic in Computer Science*.

Parametrized processes and compositional learning

Cruttwell, G. S. H., Bruno Gavranovic, Neil Ghani, Paul Wilson, and Fabio Zanasi. 2022. “Categorical Foundations of Gradient-Based Learning.” In *Programming Languages and Systems: ESOP 2022*.

Fong, Brendan, David Spivak, and Remy Tuyeras. 2019. “Backprop as Functor: A Compositional Perspective on Supervised Learning.” In *Proceedings of LICS 2019*.

Moggi, Eugenio. 1991. “Notions of Computation and Monads.” *Information and Computation* 93 (1): 55-92.

Hughes, John. 2000. “Generalising Monads to Arrows.” *Science of Computer Programming* 37 (1-3): 67-111.

McBride, Conor, and Ross Paterson. 2008. “Applicative Programming with Effects.” *Journal of Functional Programming* 18 (1): 1-13.

Evaluation, games, and adaptive control

St. Clere Smithe, Toby. 2021. “Bayesian Lenses.” arXiv:2109.04461.

St. Clere Smithe, Toby. 2023. “Approximate Inference via Fibrations of Statistical Games.” arXiv:2306.17009.

Ghani, Neil, Jules Hedges, Viktor Winschel, and Philipp Zahn. 2018. “Compositional Game Theory.” In *Proceedings of LICS 2018*.

Capucci, Matteo, Bruno Gavranovic, Jules Hedges, and Eigil Fjeldgren Rischel. 2022. “Towards Foundations of Categorical Cybernetics.” *Electronic Proceedings in Theoretical Computer Science* 372.

Open systems and architecture

Fong, Brendan. 2015. “Decorated Cospans.” *Theory and Applications of Categories* 30: 1096-1120.

Baez, John C., and Kenny Courser. 2020. “Structured Cospans.” *Theory and Applications of Categories* 35: 839-897.

Fong, Brendan, and David I. Spivak. 2019. *An Invitation to Applied Category Theory: Seven Sketches in Compositionality*. Cambridge University Press.

Plotkin, Gordon D. 1981/2004. “A Structural Approach to Operational Semantics.” *The Journal of Logic and Algebraic Programming* 60-61: 17-139.

Kahn, Gilles. 1974. “The Semantics of a Simple Language for Parallel Programming.” In *Information Processing* 74.

Retrieval and RAG

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, et al. 2020. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems* 33.

Karpukhin, Vladimir, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. “Dense Passage Retrieval for Open-Domain Question Answering.” In *Proceedings of EMNLP 2020*.

- Robertson, Stephen, and Hugo Zaragoza. 2009. “The Probabilistic Relevance Framework: BM25 and Beyond.” *Foundations and Trends in Information Retrieval* 3 (4): 333-389.
- Cormack, Gordon V., Charles L. A. Clarke, and Stefan Buettcher. 2009. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.” In *Proceedings of SIGIR 2009*.
- Jarvelin, Kalervo, and Jaana Kekalainen. 2002. “Cumulated Gain-Based Evaluation of IR Techniques.” *ACM Transactions on Information Systems* 20 (4): 422-446.
- Malkov, Yu. A., and D. A. Yashunin. 2020. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4): 824-836.
- Thakur, Nandan, Nils Reimers, Andreas Ruckle, Abhishek Srivastava, and Iryna Gurevych. 2021. “BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models.” In *NeurIPS Datasets and Benchmarks*.
- Es, Shahul, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2024. “RAGAS: Automated Evaluation of Retrieval Augmented Generation.” In *Proceedings of EACL 2024: System Demonstrations*.
- Saad-Falcon, Jon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. “ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems.” In *Proceedings of NAACL-HLT 2024*.

Verification and stateful systems

- Claessen, Koen, and John Hughes. 2000. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” In *Proceedings of ICFP 2000*.
- Lamport, Leslie. 1994. “The Temporal Logic of Actions.” *ACM Transactions on Programming Languages and Systems* 16 (3): 872-923.
- Lamport, Leslie. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.

Companion sandbox reference

The self-contained Go sandbox accompanies this book as `ProbOpt_Sandbox.zip`. After extraction:

```
cd probopt
go test ./...
go test -race ./...
go run ./cmd/probopt-demo -out out -seed 20260809 -repeats 5
```

The most useful reading order is:

1. `core/spec.go`, `seed.go`, `outcome.go`, `trace.go`, and `kernel.go`;
2. `finite/prob.go`, `dist.go`, `kernel.go`, and `laws.go`;
3. `plugin/plugin.go` and `policy.go`;
4. `experiment/types.go`, `runner.go`, and `ledger.go`;
5. `evidence/metric.go`, `paired.go`, `gate.go`, and `pareto.go`;
6. `campaign/interfaces.go`, `candidate.go`, and `run.go`;
7. `ragtoy/types.go`, `plugins.go`, `builder.go`, `search.go`, and `evaluator.go`;
8. `cmd/probopt-demo/main.go`.

The sandbox is intentionally compact. Its purpose is to make the semantics executable and inspectable, not to provide distributed scheduling, provider SDKs, production release activation, or a general measure-theoretic probability library.